

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

ML Through the Looking Glass

Machine Learning with Lenses in Haskell

Author:

Alexandre Noel

Supervisor:

Prof. Nicolas Wu

submitted for the degree of

MEng Computing (Artificial Intelligence and Machine Learning)

June 20, 2026

Abstract

We present a Haskell implementation of the parametric lens framework for gradient-based machine learning proposed by Cruttwell et al. In this framework every component of a training pipeline—layer, loss function, and optimiser—is a *parametric lens*: a composable unit carrying a forward pass and a manually derived backward pass as a single algebraic structure. Composition of lenses corresponds directly to the wiring of modules in a computation graph, and the combined parameter type is inferred automatically by the type system.

The implementation extends the original Python proof-of-concept in several directions. Tensor shapes, mini-batch sizes, compute devices, and element dtypes are encoded as GHC type-level parameters, so architectural mismatches are compile-time errors rather than runtime failures; all models are written in batched form from the ground up, whereas the original processes a single example at a time. Autoencoders are expressed as two ordinary lens pipelines composed end-to-end, demonstrating that the framework is not limited to discriminative tasks. Residual skip connections are realised as a higher-order combinator `skipPara`, built entirely from existing lens primitives, with the identity gradient path emerging automatically from the combinator structure rather than from a separate derivation. Scaled dot-product self-attention and its multi-head generalisation are implemented as `ParaLens`’ values with fully hand-derived backward passes, including the rank-one Jacobian correction for the softmax non-linearity, with the head-count and embedding-dimension compatibility constraint enforced at the type level. Variable-depth networks are constructed at the type level by typeclass induction over type-level Peano naturals. Zero overhead is verified by GHC Core inspection: the entire lens abstraction—composition, reparametrisation, the van Laarhoven `forall`—is absent from the optimised output. Every hand-derived backward pass is cross-validated against PyTorch autograd to a tolerance of 10^{-4} , so the gradients are written by hand but checked by machine. On small-scale CPU benchmarks, where tape-construction and dispatch overhead dominate over kernel compute, the framework is $5.3\times$ faster than dynamic autograd: the tape is eliminated entirely, and the hand-derived backward pass reduces to the same primitive tensor calls as the forward pass.

Contents

1	Introduction	3
1.1	The Need for a Unifying Perspective	3
1.1.1	Structural Fragmentation	3
1.1.2	Parametric Lenses	3
1.1.3	Modularity through Composition	4
1.1.4	The Cost of Dynamic Differentiation	5
1.1.5	Implementation in Haskell	5
1.2	Contributions	6
1.3	Thesis Outline	7
2	Background	8
2.1	Functional Programming in Haskell	8
2.2	Neural Networks and Gradient-Based Learning	10
2.3	Cartesian Reverse Differential Categories	11
2.4	Lenses and Optics	11
2.4.1	Origins	12
2.4.2	Van Laarhoven Lenses	12
2.4.3	Composition and Modularity	13
2.4.4	Final Note	14
2.5	Related Work	14
3	Core Implementation	17
3.1	From Lenses to Parametric Lenses	17
3.2	Wire Diagrams	17
3.3	The <i>ParaLens</i> Type	18
3.4	Lifting Ordinary Lenses	19
3.5	The Monoidal Product	19
3.6	Composition	20
3.7	Reparametrisation	21
3.8	Efficiency Considerations	22
4	Layer Implementation	23

4.1	Linear Layer and Bias	23
4.2	Activation Functions	24
4.3	Convolution	25
4.4	Pooling and Shape	26
4.5	Skip Connections	27
4.6	Attention	28
5	Loss Functions	32
5.1	Losses, Caps, and the Learning Rate	32
5.2	The Gradient Scaling Convention	33
5.3	Mean-Squared Error: <code>lossSmooth</code>	33
5.4	Softmax Cross-Entropy: <code>softMaxCELoss</code>	34
6	Optimisers	36
6.1	Gradient Descent	36
6.2	Momentum	37
6.3	Adaptive Methods	37
6.4	Per-layer Optimisers	39
7	Case Studies and Evaluation	40
7.1	Iris Flower Classification	40
7.2	MNIST Digit Recognition	45
7.3	Residual MNIST	48
7.4	MNIST Autoencoder	51
7.5	Performance Evaluation	53
7.6	Gradient Correctness Tests	54
8	Conclusion	56
8.1	Summary	56
8.2	Future Work	57
8.3	Limitations	57
8.4	Broader Impact	60
9	Declarations	61
9.1	Generative AI	61
9.2	Ethical Considerations	61
9.3	Sustainability	61
A	Hasktorch Typed-API Bug Workarounds	62

Chapter 1

Introduction

1.1 The Need for a Unifying Perspective

The dominant paradigm in modern artificial intelligence, particularly deep learning, relies heavily on gradient-based optimisation techniques [1]. While these methods have achieved remarkable success across various scientific and technological domains, the ever-increasing complexity of both models and their underpinning algorithms come hand in hand with an increasing need for explainability and rigorous analysis. Current interpretations typically rely on heuristic combinations of distinct algorithmic components, often lacking a thorough, unified mathematical foundation that can scale with the sophistication of modern models [2].

1.1.1 Structural Fragmentation

A typical supervised learning scenario orchestrates an interaction between three seemingly distinct entities: the model, the loss map, and the optimiser [3]. In standard practice, these components are treated as independent modules: one might swap Softmax cross-entropy for focal loss, or replace basic gradient descent with adaptive methods like Adam or Nesterov momentum [4, 5].

However, this modularity is often implemented operationally rather than mathematically. Questions regarding the shared structural properties of these components, and whether they can be described by a single uniform language, remain largely unaddressed in standard frameworks. Furthermore, standard approaches usually restrict learning to continuous domains (smooth maps), often failing to generalise to discrete settings such as Boolean circuits [4].

1.1.2 Parametric Lenses

This project explores a unifying semantic framework proposed by Cruttwell et al. [4], which posits that gradient-based learning is fundamentally a composition of parametric lenses. In this view, a lens serves as a formal interface between internal parameters and external observables. It provides a mechanism to “zoom” into specific components of a

model, adjust their internal states, and propagate those changes back through the system.

This perspective abstracts the learning process into three core principles:

1. **Parameterisation:** All components—from neural weights to loss functions—are viewed as parameterised maps [4]. This lifts standard functions into a space where their internal configurations are first-class citizens.
2. **Bidirectional Information Flow:** The architecture must support both the forward propagation of predictions and the backward propagation of updates. This is captured by the categorical structure of lenses [6], where the “view” (forward pass) and “update” (backward pass) are intrinsically linked.
3. **Abstract Differentiation:** Parameter updates are driven by differentiation, modeled via Cartesian Reverse Differential Categories (CRDCs) [7].

By viewing the entire training loop as a morphism in a structured category, we move away from “black box” optimisation. Instead, the optimiser is reinterpreted not as an external control loop, but as a specific type of lens that reparameterises the model, creating a clean algebraic structure for gradient flows.

1.1.3 Modularity through Composition

The power of this formulation lies in its compositionality. Composing lenses corresponds to the physical “wiring” of sub-modules within a model. When we compose two parametric lenses, we are effectively defining how gradients should flow across the boundaries of different layers or loss functions. Crucially, the author of each lens writes only its *local* backward pass; composition assembles these local passes into the global chain rule automatically, and the combined parameter type is inferred rather than declared. This is what distinguishes the approach from hand-rolled backpropagation, where the wiring between layers must itself be written and maintained by hand.

This approach offers significant advantages over standard modularity:

- **Recursive Scaling:** Large-scale architectures can be treated as single lenses composed of smaller, verified lenses, allowing for a rigorous analysis of complex systems through their constituent parts.
- **Internalised Optimisation and Deep Compositionality:** In standard practice, an optimiser is an external control loop that sits outside the model. In this framework, an optimiser is reinterpreted as a reparameterisation lens. This allows for “Deep Compositionality”: one can easily “wire” different optimisers into differ-

ent sub-modules of a single architecture (e.g., using an Adam-lens for convolutional layers and an SGD-lens for dense layers). The resulting hybrid structure is itself a single lens, abstracting away internal complexity and presenting a uniform interface to the rest of the system.

1.1.4 The Cost of Dynamic Differentiation

Reverse-mode automatic differentiation, as provided by frameworks such as PyTorch, makes this convenient: the user writes only the forward pass, and the backward pass is built automatically by recording each operation onto a *tape* and replaying it in reverse. The convenience has a structural cost. The tape is rebuilt on every step, so each pass pays for graph-node allocation and dynamic dispatch, and shapes are checked only when a kernel runs, so an architectural mismatch surfaces as a runtime exception rather than a rejected program. The parametric lens framework sits at the opposite end: each lens carries its own backward pass, so there is no tape and the chain rule is discharged structurally by composition, but the cost is paid up front by the library author, who must derive every backward pass by hand. The central question of this thesis is whether a sufficiently expressive host language can make that trade pay: whether the hand-derived passes can be *verified*, so that surrendering automatic differentiation introduces no silent gradient bugs, and *compiled away*, so that the abstraction imposes no runtime penalty over hand-written code.

1.1.5 Implementation in Haskell

The objective of this project is to instantiate the Cruttwell et al. framework in Haskell. Previous implementations have used Python [4]; Haskell offers a strong type system and mature libraries for optics [8, 9] that make the type-level guarantees central to this thesis expressible directly in the language. Two features of the language do the decisive work. The van Laarhoven encoding of lenses (Chapter 2) represents a lens as a single higher-rank function, so that composition is ordinary function composition and the whole abstraction collapses to direct calls once the compiler fixes the functor at each use site; this is what makes the zero-overhead result attainable. And PyTorch’s autograd, rather than being a runtime dependency, is repurposed as a *testing oracle*: every hand-derived backward pass is cross-validated against it to a tolerance of 10^{-4} , so the gradients are written by hand but checked by machine.

This thesis asks: can the Cruttwell et al. framework be realised in Haskell with zero abstraction overhead and static architectural safety, and at what cost to expressiveness? The question is one of foundations rather than scale: the aim is to establish that the categorical structure can be carried into a real language with its guarantees intact, and to

map where those guarantees end. The library is accordingly built against representative models (classifiers, an autoencoder, attention) rather than production-scale ones, and Chapter 8 marks the boundary of the current design: the stateful and stochastic machinery (dropout, batch normalisation, global optimiser state) that the lens model does not yet reach.

1.2 Contributions

This thesis makes the following original contributions relative to the existing literature, and in particular to the Python proof-of-concept that accompanies Cruttwell et al. [4]:

1. **Static type safety for tensor computation.** Tensor shapes, batch sizes, devices, and dtypes are encoded as type-level parameters via `DataKinds` and constraint synonyms, so shape, batch, and device/dtype mismatches are compile-time errors; the original Python implementation performs no such checking.
2. **Type-level variable network depth.** The `Stack.hs` module uses Peano naturals and typeclass induction to build networks of depth n whose parameter type is a fully concrete nested tuple at each n , preserving GHC’s ability to specialise and unbox the entire parameter structure with zero overhead relative to a hand-written network of the same depth.
3. **Extended architecture support.** The framework reaches beyond the MLP scope of the original paper: autoencoders are ordinary *ParaLens*’ pipelines composed end-to-end via $(\bullet)$ with the bottleneck enforced at the type level, and residual skip connections are a single higher-order combinator *skipPara* built from existing primitives, with the identity gradient path emerging automatically from the combinator structure.
4. **Attention as a *ParaLens*’.** Scaled dot-product self-attention and its multi-head generalisation are implemented with fully hand-derived backward passes, including the rank-one Jacobian correction for the softmax non-linearity. The constraint $e = h \cdot hd$ is enforced at the type level, so a head-count or embedding-dimension mismatch is a compile-time error rather than a silent shape failure at runtime.
5. **Zero-overhead evidence via GHC Core.** Compiling the lens-based forward pass and an equivalent hand-written one with `-O2 -ddump-simpl` yields structurally identical worker functions: the entire *ParaLens* abstraction (composition, reparametrisation, the van Laarhoven `forall`) is absent from the optimised output.

1.3 Thesis Outline

Chapter 2 covers the background: Haskell’s type system, optics (including the van Laarhoven representation), gradient-based learning, and Cartesian Reverse Differential Categories. Chapter 3 defines the **ParaLens** type, its composition operator ($\bullet$), and the **repara** combinator. Chapters 4, 5, and 6 cover layers, loss functions, and optimisers; Chapter 7 presents the Iris and MNIST case studies, the GHC Core zero-overhead evidence, and the type-level depth experiment; and Chapter 8 concludes.

Source code. The full implementation is available at https://github.com/LiquidPulsar/optics_haskell/tree/static.

Chapter 2

Background

This chapter introduces the three bodies of knowledge that underpin the thesis: functional programming in Haskell, including the advanced type system features that enable static shape checking; the theory of optics that supplies the mathematical objects for representing neural network layers; and the fundamentals of gradient-based learning together with the categorical formalism used to reason about it.

2.1 Functional Programming in Haskell

Haskell is a purely functional, statically typed language with lazy evaluation. *Pure* functions have no side effects: a function $f :: a \rightarrow b$ maps every value of type a to a value of type b without reading or writing global state. Purity makes programs compositional by default: the behaviour of a composition $f \circ g$ is fully determined by the behaviours of f and g individually, which is the property the framework exploits to build large networks from small verified components. Five advanced type-system features support the framework: type classes for overloaded interfaces; the kind system and *DataKinds* for lifting tensor shapes to the type level; type families for computing output shapes at compile time; higher-rank polymorphism for composable lenses; and constraint synonyms for keeping signatures readable.

Type Classes

The primary abstraction mechanism is the *type class*: a class declares an interface and instances provide concrete implementations. Type class resolution is purely compile-time: GHC selects and inlines the appropriate instance, leaving no dispatch overhead at runtime. This property is crucial in Section 7.1, where the entire lens abstraction is shown to vanish in the compiled output.

The Kind System and DataKinds

Every type in Haskell has a *kind*, which is the “type of a type.” Ordinary types such as *Int* have kind *Type*. A type constructor such as *Maybe* has kind $Type \rightarrow Type$: it takes one type argument before producing a concrete type.

The `{-# LANGUAGE DataKinds #-}` extension promotes value-level data constructors to the type level. The standard natural numbers, for instance, are promoted to kind `Nat`, and lists of naturals to kind `[Nat]`. The typed tensor library `Hasktorch` exploits this directly: a tensor on device `dv`, with element type `dt`, and shape `[m, n]` is written:

```
example :: T.Tensor dv dt [m, n]
```

All three properties: device `dv`, element dtype `dt`, and shape `[m, n]`; are resolved at compile time. An attempt to multiply a `[3, 4]` matrix by a `[5, 6]` matrix produces a type error before any code runs. No runtime shape assertions are needed. Throughout this thesis, type-level lists of `Nat` appear wherever tensor shapes are tracked: kernel sizes `[kH, kW]`, convolution output dimensions, and batch sizes `[b, d]` are all compile-time entities.

Type Families

A *type family* is a type-level function computed by the compiler.

The `{-# LANGUAGE TypeFamilies #-}` extension enables pattern-matching on types in closed equations:

```
type family ToPeano (n :: Nat) :: PeanoNat where
  ToPeano 1 = One
  ToPeano n = Succ (ToPeano (n - 1))
```

The compiler reduces `ToPeano 3` to `Succ (Succ One)` at compile time, with no runtime cost. Type families appear throughout the framework for computing output shapes from layer parameters: the output height of a convolution with kernel `k`, stride `s`, and padding `p` applied to an input of height `h` is $\lfloor (h - k + 2p) / s \rfloor + 1$, and this computation is performed by a type family so that the output shape is available for type-checking the next layer.

Higher-Rank Polymorphism

Standard Haskell polymorphism places the $\forall$ at the outermost level of a type: `id ::  $\forall a \circ a \rightarrow a$` . The `{-# LANGUAGE RankNTypes #-}` extension allows the quantifier to appear inside a type:

```
type Lens s t a b =  $\forall f \circ Functor\ f \Rightarrow (a \rightarrow f\ b) \rightarrow s \rightarrow f\ t$ 
```

Here $\forall f$ is scoped inside the synonym: any value of type `Lens s t a b` must work for *any* `Functor` the caller chooses. This is the van Laarhoven representation described in Section 2.4.2. The higher-rank quantifier is the reason lenses compose with plain (`o`): the same function composes for any functor; and is also the mechanism by which the lens abstraction collapses to direct function calls when the functor is fixed at the call site (Section 7.1).

Constraint Synonyms

The `{-# LANGUAGE ConstraintKinds #-}` extension allows the kind *Constraint* (the kind of type class requirements) to be treated as a first-class kind. This enables *constraint synonyms*:

```
type SaneDT dv dt =
  ( T.KnownDType dt
    , T.StandardFloatingPointDTypeValidation dv dt
    , CanMMLens dv dt )
```

SaneDT dv dt is a single alias for a conjunction of three constraints. Functions that require all three can write *SaneDT dv dt* $\Rightarrow$ in their context rather than enumerating each constraint separately. Constraint synonyms keep type signatures concise, and the type checker verifies the full set when a concrete *dv* and *dt* are supplied.

2.2 Neural Networks and Gradient-Based Learning

A *neural network* is a parametric function $f_\theta : A \rightarrow B$ indexed by a weight vector $\theta \in P$. The network is composed from differentiable building blocks (affine maps, activation functions, normalisation layers), each with their own local parameters. Training adjusts θ to minimise a scalar loss $\mathcal{L}(f_\theta(x), y)$ over a dataset of labelled pairs (x, y) .

Gradient Descent

When P is a real vector space and $\mathcal{L}$ is differentiable, the canonical update rule is gradient descent:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}, \quad \eta > 0.$$

Each step moves θ in the direction of steepest loss decrease. Practical optimisers (e.g. momentum, Adam, AdaGrad) modify this rule to improve convergence, but all share the same structure: a function from current parameters and a gradient signal to updated parameters. In this framework that function is itself a lens, described in Chapter 6.

Backpropagation

For a composed model $f_\theta = f_{\theta_L}^{(L)} \circ \dots \circ f_{\theta_1}^{(1)}$, computing $\nabla_\theta \mathcal{L}$ by the chain rule is *backpropagation*. The forward pass evaluates $f_\theta(x)$ layer by layer, caching intermediate activations; the backward pass propagates a gradient signal from the output back through each layer in reverse, computing $\partial \mathcal{L} / \partial \theta_\ell$ at each step.

In this framework both passes are encoded simultaneously in a single lens: the *getter* is the forward pass and the *setter-getter* pair is the backward pass. The chain rule is realised by lens composition via $(\circ)$ and $(\bullet)$. No separate backward-pass data structure is needed.

Mini-Batching

In practice, the gradient is estimated on a *mini-batch*: a small subset of the dataset drawn at each step. If each sample has shape $[d]$, a batch of b samples has shape $[b, d]$, and all b forward and backward passes are performed simultaneously as a single tensor operation. GPU hardware is optimised for this regular parallelism, making batching essential for performance.

In this framework the batch size b is a type-level *Nat*. A model typed to accept $T.Tensor\ dv\ dt\ [b, d]$ cannot be accidentally applied to a tensor of a different batch size: the mismatch is a compile-time type error rather than a silent shape broadcast.

2.3 Cartesian Reverse Differential Categories

The theoretical foundation for backpropagation in this framework is the notion of a *Cartesian Reverse Differential Category* (CRDC) [4]. CRDCs provide an axiomatic account of reverse-mode automatic differentiation that is independent of any particular programming language or number system.

The key property used in this thesis is that every object A in a CRDC carries a *natural addition* $+_A : A \times A \rightarrow A$, arising from the abelian group structure that CRDCs impose. In the standard category of Euclidean spaces this is ordinary vector addition; in a category of boolean circuits it would be a different, discrete operation: the abstraction is independent of the specific category.

The training loop performs precisely this natural addition:

$$\theta_{\text{new}} = \theta_{\text{old}} + \partial\theta,$$

where $\partial\theta$ is whatever the backward pass emits. This is not an implementation convention but the structural update rule of the CRDC. Whether the step is gradient *descent* or *ascent*, and at what scale, is determined entirely by the sign and magnitude of $\partial\theta$, which the learning-rate cap (Section 5.1) controls by injecting a positive or negative seed into the backward pass. No modification to the update rule itself is required to switch between the two modes.

2.4 Lenses and Optics

Compound data structures are ubiquitous in modern programming, and their inherent modularity is key to building and reasoning with complex ensembles. Lenses[10] are one approach to solve the problem of accessing the components of such structures, also known as the *view-update problem*[11].

2.4.1 Origins

A simple lens from outer type S to inner type A (which we shall denote as $Lens' s a$ for reasons that will become apparent) can be thought of as comprising[10] a function $view :: s \rightarrow a$ which projects the component A , and a function $update :: s \rightarrow a \rightarrow s$ which accepts a new value of A and uses it to modify a given S .

```
data  $Lens' s a = Lens' \{ view :: s \rightarrow a, update :: s \rightarrow a \rightarrow s \}$ 
```

As a concrete example, consider the relationship between the following *Account* and *PhoneNumber* data types:

```
newtype  $PhoneNumber = PhoneNumber Int$ 
newtype  $Account = Account$ 
  {  $phone :: PhoneNumber$ 
    ,  $accountId :: Int$ 
  }
 $acctToPhone :: Lens' Account PhoneNumber$ 
 $acctToPhone = Lens' view update$ 
where
   $view :: Account \rightarrow PhoneNumber$ 
   $view = phone$ 
   $update :: Account \rightarrow PhoneNumber \rightarrow Account$ 
   $update acct num = acct \{ phone = num \}$ 
```

2.4.2 Van Laarhoven Lenses

It is tempting to extend our *Lens* type with a modification function $mod :: (a \rightarrow a) \rightarrow s \rightarrow s$ [12]. This begs the question: at what point do we stop? We could usefully include $modM :: (a \rightarrow Maybe a) \rightarrow s \rightarrow Maybe s$, or even $modIO :: (a \rightarrow IO a) \rightarrow s \rightarrow IO s$. Before letting this get out of hand, we observe that both share the structure of a *Functor*, yielding $modF :: Functor f \Rightarrow (a \rightarrow f a) \rightarrow s \rightarrow f s$.

As a quick refresher, the category of *Functors*[13, 14] supports the lifting of an arbitrary function into their type. In Haskell terms:

```
class  $Functor f$  where
   $fmap :: (a \rightarrow b) \rightarrow f a \rightarrow f b$ 
```

By careful choice of specific *Functors* we can recover *view*, *mod*, and *update*. Note that providing *const a* to *mod* recovers *update*, so we need only produce the first two. The key choices are [15, 16]:

```

newtype Identity a  = Identity { runIdentity :: a }
newtype Const  a b = Const  { getConst    :: a }

instance Functor Identity where
    fmap f (Identity x) = Identity (f x)

instance Functor (Const m) where
    fmap _ (Const v) = Const v

```

Applying *Identity* to *modF* yields *modF* :: $(a \rightarrow \text{Identity } a) \rightarrow s \rightarrow \text{Identity } s$, isomorphic to *mod*! All that is left is wrapping and unwrapping the *Identity*¹.

Applying *Const a* to *modF* yields *modF* :: $(a \rightarrow \text{Const } a \ a) \rightarrow s \rightarrow \text{Const } a \ s$, isomorphic to $(a \rightarrow a) \rightarrow s \rightarrow a$, which when provided *id* is exactly *view*.

Thus a *Lens'* only needs *modF*, and we can simplify to the type alias:

```

type Lens' s a =  $\forall f \circ \text{Functor } f \Rightarrow (a \rightarrow f \ a) \rightarrow s \rightarrow f \ s$ 

```

This representation also lets GHC apply aggressive optimisations for higher-order functions, which will matter as we compose increasingly complex lenses.

It is often convenient to allow the types in the reverse direction to differ from those in the forward direction, yielding *update* :: $s \rightarrow b \rightarrow t$ for fresh types *B* and *T*:

```

data Lens s t a b = Lens { view :: s → a, update :: s → b → t }

```

In the Van Laarhoven style [9]:

```

type Lens s t a b =  $\forall f \circ \text{Functor } f \Rightarrow (a \rightarrow f \ b) \rightarrow s \rightarrow f \ t$ 
type Lens' s a    = Lens s s a a

lens :: (s → a) → (s → b → t) → Lens s t a b
-- Note: fmap written infix
lens sa sbt afb s = sbt s <$> afb (sa s)

```

2.4.3 Composition and Modularity

The Van Laarhoven encoding gives composition for free. Given:

```

x :: Lens s t a b  -- forall f . Functor f => (a -> f b) -> (s -> f t)
y :: Lens a b c d  -- forall f . Functor f => (c -> f d) -> (a -> f b)

```

Standard function composition ($\circ$) has type $(b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$, so $x \circ y$ unifies immediately:

¹This has no runtime cost thanks to the semantics of **newtype** and **coerce**[17].

$$x \circ y :: \text{Lens } s \ t \ c \ d \quad \text{-- forall } f . \text{ Functor } f \Rightarrow (c \rightarrow f \ d) \rightarrow (s \rightarrow f \ t)$$

No glue code required: composing lenses with $(\circ)$ is simply function composition. This allows compound accessors to be built from primitives without any overhead.

2.4.4 Final Note

So far we have considered only lenses whose focus is a concrete field of a record, as with *PhoneNumber* inside *Account*. This is not a requirement, and in subsequent sections we will see that machine learning layers can themselves be expressed as more exotic lenses.

2.5 Related Work

Cruttwell et al., 2022

The categorical foundations of the framework are due to Cruttwell et al. [4], who showed that gradient-based learning is naturally modelled as composition in a category $\mathbf{Para}(C)$ of parametric morphisms over a CRDC C . They define parametric lenses, the composition rule, and the connection between backpropagation and the CRDC reverse derivative, and demonstrate the framework on small MLP examples.

The accompanying implementation is intentionally minimal: it is dynamically typed, operates on single examples rather than batches, and covers only dense layers and convolutions.

Importantly, the implementation in Python (whilst a natural choice for ease of development) does not fulfil the original intentions of the lens design in terms of efficient composition. Python imposes significant overhead due to the additional function calls required for composing our lenses, which cannot be optimised out.

Haskell offers far better support for lenses: thanks to its optimising compiler and powerful type reasoning capabilities we can eliminate all overhead vs hand-written code. The present work instantiates the prior categorical structure in Haskell, contributing: statically typed tensor shapes via *DataKinds*; device and dtype polymorphism; batching as a type-level parameter; autoencoders and attention mechanisms; type-level variable network depth and zero-overhead evidence via *GHC Core*. These contributions are discussed in detail in Chapter 7.

Backprop as a Functor

Fong et al. [5] independently proposed viewing backpropagation as a functor between categories of learners. Their work establishes similar compositionality results for supervised learning but takes a different categorical approach: composition of learners corresponds

directly to composition of update rules, without the explicit Para construction. Cruttwell et al. unify and generalise both lines of work within the CRDC framework.

Grenade

Grenade [18] is a dependently-typed neural network library for Haskell whose `Network layers shapes` type encodes both layer types and intermediate tensor shapes at the type level. It shares the present work’s goal of compile-time architectural checking, but has three limitations that motivated a fresh approach: the intermediate shape list must be written by hand (the present framework infers it automatically from $(\bullet)$ composition); it is built on `hmatrix` [19], which is restricted to one- and two-dimensional arrays and therefore cannot track shapes for convolution or attention tensors natively; and it provides no GPU backend. The repository has not received substantive updates since 2022.

The `backprop` Library

Le’s `backprop` [20] library brings reverse-mode automatic differentiation to Haskell by tracing the forward computation at runtime and constructing a dynamic graph from which gradients are derived. The approach is analogous to PyTorch’s autograd: the programmer writes only the forward pass, and the backward pass is produced automatically.

One might ask whether `backprop` could serve as the backward-pass mechanism within a lens framework, reducing the lens setter to a derived artefact. This would be self-defeating. A Van Laarhoven lens is already a single higher-rank function— $\forall f \circ Functor\ f \Rightarrow (a \rightarrow f\ b) \rightarrow s \rightarrow f\ t$ —not a pair of functions stored together. The getter and setter are both specialisations of this one polymorphic function via $f = Const$ and $f = Identity$; the “hold one function rather than two” goal is already achieved by the lens itself. The lens setter is not a convenience wrapper around a gradient computation; it *is* the gradient computation, statically encoded. Replacing it with a traced tape would reintroduce the runtime overhead the framework is designed to eliminate: the backward pass of a Van Laarhoven lens collapses to a direct chain of tensor operations at compile time (Section 7.1), whereas a dynamic graph cannot be inlined away.

The deeper distinction is categorical. In this framework the update rule $\theta \leftarrow \theta + \partial\theta$ is the CRDC natural addition, not a gradient-specific operation; the sign and magnitude of $\partial\theta$ are controlled externally by the learning-rate cap, and the same structural update rule applies in any CRDC, including discrete ones. `backprop` is tied to differentiable, real-valued functions and has no account of this generality. The two libraries therefore solve different problems: `backprop` makes gradient computation convenient; this thesis makes the categorical structure of gradient-based learning explicit and zero-overhead.

Conventional Frameworks: PyTorch and JAX

The dominant operational frameworks treat neural networks as imperative programs annotated with automatic differentiation. In define-by-run systems such as PyTorch’s eager mode, gradients are computed by tracing the forward computation graph afresh on each step and applying reverse-mode AD.

This differs from the categorical approach in three respects. First, the optimiser is a global object that maintains state separately from model parameters; mixing update rules across layers requires explicit parameter grouping. In this framework the optimiser state is embedded into the parameter type via *repara*, and per-layer optimisers arise automatically from the product structure of $(\bullet)$. Second, tensor shape errors typically surface at runtime when tensors are first combined; the present framework reports them as type errors at compile time. Third, there is no clear categorical account of what the composition of two PyTorch modules means mathematically; here, composition is literally morphism composition in $\mathbf{Para}(C)$.

JAX warrants separate treatment, since it does not rebuild a tape per step. A jitted JAX program traces the forward function once and lowers it through XLA to a compiled, kernel-fused executable, so the per-step tracing cost this framework avoids is largely absent there as well. The distinction is therefore not raw overhead but how it is removed and what is guaranteed. JAX eliminates overhead through a separate staged pipeline: a tracer that specialises the function to concrete shapes, an intermediate *jaxpr*, and a domain-specific compiler. The lens backward pass, by contrast, is erased by GHC’s general-purpose optimiser acting on ordinary source, with no tracer and no graph intermediate representation (Section 7.1). And JAX’s shape checking is performed by that tracer when the function is specialised on concrete inputs, not by the host type system: an ill-shaped model is rejected only once it is traced, whereas here the same mismatch is a type error in the program text, caught before any code runs and independently of any input. Neither framework, finally, offers a categorical account of composition of the kind made precise above.

Hasktorch

Hasktorch is the underlying tensor library used in this implementation. It provides Haskell bindings to LibTorch (the C++ backend of PyTorch) with a typed interface: *T.Tensor dv dt shape* tracks the device, dtype, and shape at the type level, supplying the primitive operations (matrix multiply, convolution, activation functions) that the framework layers on top of. Hasktorch is not itself a categorical ML framework; it provides the typed computational substrate.

Chapter 3

Core Implementation

3.1 From Lenses to Parametric Lenses

Building on the Van Laarhoven lenses of the previous chapter, we now introduce *parametric lenses* [4]: an extension that promotes the parameter of a computation to a first-class wire in the type. A standard lens $Lens\ s\ t\ a\ b$ is stateless: it carries no mutable configuration of its own. For machine learning this is insufficient: a neural network layer holds weight matrices and bias vectors that are read during the forward pass and updated during the backward pass. Parametric lenses make these parameters structural, so that the update rule is part of the type rather than an afterthought.

3.2 Wire Diagrams

A lens $Lens\ A\ A'\ B\ B'$ has a direct operational reading as two functions: a *forward* computation $f : A \rightarrow B$ and a *backward* computation $f^* : A \times B' \rightarrow A'$. The backward pass requires the original input A , so the wire carrying A branches: one copy feeds f while the other curves to the right-hand side of f^* .

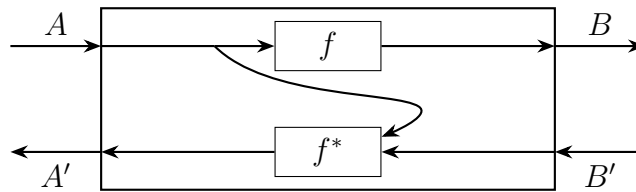


Figure 3.1: $Lens\ A\ A'\ B\ B'$ decomposed into forward pass f and backward pass f^* .

Substituting $A = (p, a)$ and $A' = (p', a')$ gives the same diagram for a $Lens\ (p, a)\ (p', a')\ b\ b'$, in which the parameter p is bundled with the data on the horizontal wire:

Separating the p component onto a dedicated *vertical* axis (p entering from above on the left and p' exiting upward on the right), and retaining only a, a' on the horizontal data wires yields the canonical wire diagram for a parametric lens:

The left and right edges carry the data wires (a and b on the upper forward channels, a'

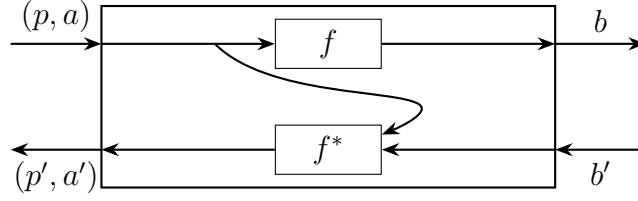


Figure 3.2: Substituting $A = (p, a)$, $A' = (p', a')$: parameter and data bundled on the horizontal wire.

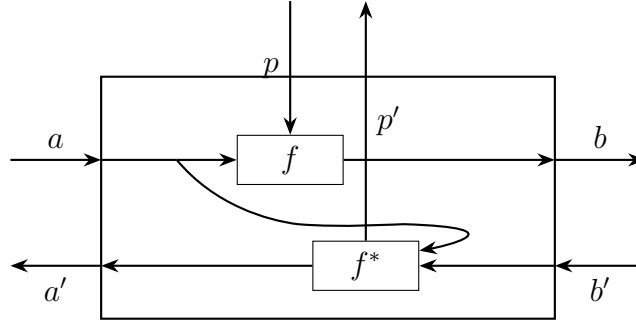


Figure 3.3: $ParaLens\ p\ p'\ a\ a'\ b\ b'$: p and p' factored onto the vertical axis.

and b' on the lower backward channels), and the top edge carries both parameter wires. Setting the Van Laarhoven functor to *Identity* recovers the forward pass $(p, a) \rightarrow b$; setting it to *Const* b' recovers the backward pass $(p, a, b') \rightarrow (p', a')$. The design therefore inherits lens composition, the lens library, and all compiler optimisations for free.

3.3 The *ParaLens* Type

The implementation realises the wire-diagram intuition as a type alias over the standard Van Laarhoven lens:

```
type ParaLens  $p\ p'\ a\ a'\ b\ b' = Lens\ (p, a)\ (p', a')\ b\ b'$ 
```

The six type variables have the following roles:

Variable	Role
p, p'	parameter type before and after the update
a, a'	data type in the forward and backward directions
b, b'	output type in the forward and backward directions

Expanding the alias, $ParaLens\ p\ p'\ a\ a'\ b\ b'$ is:

$$\forall f. Functor\ f \Rightarrow ((p, a) \rightarrow f\ b) \rightarrow (p, a) \rightarrow f\ (p', a')$$

Setting $f = Identity$ recovers the forward pass f ; setting $f = Const\ b'$ recovers the

backward pass f^* , exactly as with ordinary lenses.

The simplified variant *ParaLens'* fixes the types to be unchanged by the pass: the natural choice for layers that do not change their own interface:

```
type ParaLens' p a b = ParaLens p p a a b b
```

3.4 Lifting Ordinary Lenses

A standard lens carries no parameters at all. It can be embedded into the parametric setting by equipping it with the unit parameter $()$ via *toPara*:

```
toPara :: Lens a a' b b' → ParaLens () () a a' b b'
toPara = (leftUnit ∘)
```

The helper *leftUnit* is the isomorphism witnessing that $((), a)$ is isomorphic to a : pairing with the unit type adds no information:

```
leftUnit :: Iso ((), a) ((), a') a a'
leftUnit = iso snd ((), )
```

Here *Iso s t a b* is the van Laarhoven type for an isomorphism between (s, t) and (a, b) , exported by **Control.Lens** [9]; *iso* :: $(s \rightarrow a) \rightarrow (b \rightarrow t) \rightarrow \text{Iso } s \ t \ a \ b$ constructs one from a pair of inverse functions. *leftUnit* is built from *snd* :: $((), a) \rightarrow a$ (the forward direction) and $((),) :: a \rightarrow ((), a)$ (the backward direction).

Composing *leftUnit* on the left of any lens rearranges the source pair so that the trivial parameter slot disappears from the type. Symmetrically, *rightUnit* handles the case where the unit appears on the right:

```
rightUnit :: Iso (a, ()) (a', ()) a a'
rightUnit = iso fst (, ())
```

3.5 The Monoidal Product

Composition is not the only structural operation. Given a lens that acts on some component, it is frequently necessary to *extend* it to act on one component of a pair while leaving the other untouched. This is the role of *rightLens* and *leftLens*:

```
rightLens :: Lens p p' q q' → Lens (a, p) (a, p') (a, q) (a, q')
rightLens = alongside id

leftLens :: Lens p p' q q' → Lens (p, a) (p', a) (q, a) (q', a)
leftLens = flip alongside id
```

These are the two injections of the monoidal product: *alongside* from **Control.Lens** pairs two lenses into a lens on a product type. Passing *id* on one side leaves that component untouched. In wire-diagram terms, *rightLens l* adds an *extra wire on the left* that passes through the box unchanged:

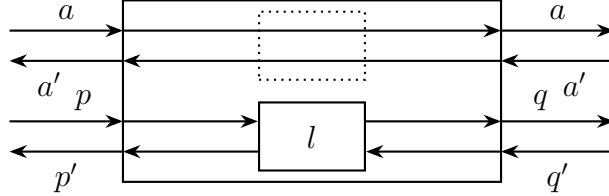


Figure 3.4: *rightLens l* (outer box): *id* passes *a* and *a'* through unchanged, while *l* transforms $p \rightarrow q$ forward and $q' \rightarrow p'$ backward.

3.6 Composition

The central operation is sequential composition, written $(\bullet)$. Given two parametric lenses whose data types are compatible:

$$f :: \text{ParaLens } p \ p' \ a \ a' \ b \ b'$$

$$g :: \text{ParaLens } q \ q' \ b \ b' \ c \ c'$$

their composition $f \bullet g$ threads the output of *f* into the input of *g*, and *pairs* the two parameter wires into a single product wire:

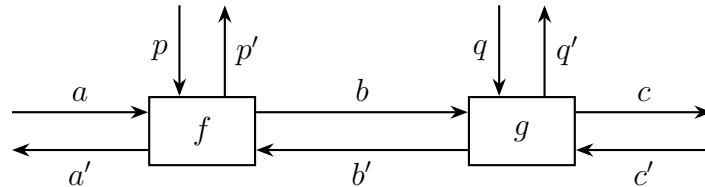


Figure 3.5: Composition $f \bullet g :: \text{ParaLens } (p, q) \ (p', q') \ a \ a' \ c \ c'$. The bidirectional middle channel shows *b* (forward, upper) and *b'* (backward, lower) flowing in opposite directions between the two layers.

The result has type $\text{ParaLens } (p, q) \ (p', q') \ a \ a' \ c \ c'$: the combined parameter is the *product* of the two individual parameter types. This directly models backpropagation: during the backward pass, gradients flow through *g* and then *f*, with each layer independently updating its own slice of the combined parameter.

The implementation uses three auxiliary isomorphisms to reassemble the nested pairs into the required shape:

infixr 8 •

$$(\bullet) :: \text{ParaLens } p \ p' \ a \ a' \ b \ b' \rightarrow \text{ParaLens } q \ q' \ b \ b' \ c \ c' \rightarrow \text{ParaLens } (p, q) \ (p', q') \ a \ a' \ c \ c'$$

$$f \bullet g = \text{swapFst} \circ \text{rotate} \circ \text{rightLens } f \circ g$$

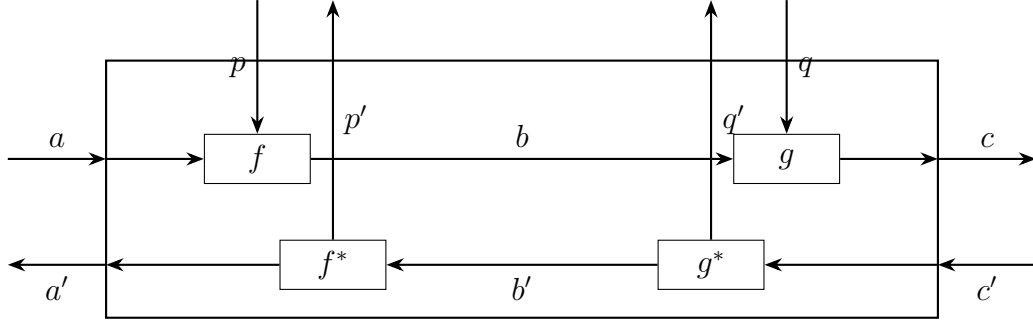


Figure 3.6: $f \bullet g$ as a single *ParaLens* $(p, q) (p', q') a a' c c'$: each sub-lens carries its own independent parameter channel on the vertical axis, and the intermediate wire b/b' links the two halves.

Reading right-to-left, and tracking the source type at each step:

1. $g :: \text{Lens } (q, b) (q', b') c c'$. This is the inner parametric lens mapping b to c while carrying q as a parameter.
2. $\text{rightLens } f$ extends $f :: \text{Lens } (p, a) (p', a') b b'$ to act on the right slot of a pair with q on the left. Composed with g :
 $\text{rightLens } f \circ g :: \text{Lens } (q, (p, a)) (q', (p', a')) c c'$
3. $\text{rotate} :: \text{Iso } ((a, b), c) \rightarrow (a, (b, c))$ – reassociates the nested triple, changing the source from $(q, (p, a))$ to $((q, p), a)$.
4. $\text{swapFst} :: \text{Iso } ((a, b), c) \rightarrow ((b, a), c)$ – swaps the inner pair, changing $((q, p), a)$ to $((p, q), a)$.

The final source type is $((p, q), a)$, which matches *ParaLens* $(p, q) (p', q') a a' c c'$ as required.

3.7 Reparametrisation

Given a lens $r :: \text{Lens } q q' p p'$ that relates two parameter spaces, we can *reparametrise* any *ParaLens* $p p' a a' b b'$ to run in the q parameter space:

$$\begin{aligned} \text{repara} &:: \text{Lens } q q' p p' \rightarrow \text{ParaLens } p p' a a' b b' \rightarrow \text{ParaLens } q q' a a' b b' \\ \text{repara } r &= (\text{leftLens } r \circ) \end{aligned}$$

This is the mechanism by which an *optimiser* (itself a lens that maps raw gradients to updated parameters) can be wired into a model without changing the model’s own structure. Composing $\text{leftLens } r$ on the left redirects the parameter wires through r before they reach the model, so the model continues to “see” its own parameter type while the outer context handles the optimiser state.

3.8 Efficiency Considerations

At first glance, the *ParaLens* type appears to introduce a lot of extra pairing and unpairing of parameters, which could be a source of overhead. However, the implementation relies on the fact that these operations are *isomorphisms* that can be optimised away by the compiler. The *swapFst*, *rotate*, and *rightLens* functions are all built from simple tuple manipulations that the Haskell compiler can inline and eliminate, resulting in no runtime overhead for the composition operation. This means that the elegant compositional structure of *ParaLens* does not come at the cost of performance, and the abstraction can be used freely without worrying about efficiency penalties.

A further efficiency observation concerns the training loop itself. During a training step only the setter is invoked (instantiating $f = \text{Const } b'$) which internally re-executes the forward computation before applying the backward computation. The compiler therefore sees a *single closed function* $(\theta, a, b') \rightarrow (\theta', a')$ rather than two separate phases coordinated via a runtime tape. This gives GHC the opportunity to apply optimisations—common subexpression elimination, dead-code elimination, and cross-boundary inlining—across the combined forward and backward computation simultaneously, opportunities that are structurally unavailable to dynamic-graph frameworks where the tape acts as an opaque barrier between the two phases.

Chapter 4

Layer Implementation

Each concrete neural network layer is a term of type *ParaLens'* or *Lens'*, directly instantiating the wire diagram of Section 3.2. The guiding principle is simple: *stateless* transformations, those with no learnable parameters, are plain *Lens'* values with no vertical parameter wires; *stateful* ones are *ParaLens'* values whose parameter type holds the learnable tensors. This distinction is enforced at the type level and is not a matter of convention.

4.1 Linear Layer and Bias

The linear layer is the framework's core building block; its backward pass establishes the gradient-flow pattern (getter computes the forward output, setter returns gradients for both weights and inputs) that every subsequent layer follows.

The weight matrix alone constitutes a *ParaLens'* whose parameter is a typed tensor of shape $[o, i]$:

```
linear :: T.MatMulDTypeIsValid dv dt
      => ParaLens' (Tensor dv dt [o, i]) (Tensor dv dt [batch, i]) (Tensor dv dt [batch, o])
linear = lens fwd rev
where
  fwd (w, x) = T.matmul x (transp w)
  rev (w, x) grad = (T.matmul (transp grad) x, T.matmul grad w)
```

The forward pass computes $xW^\top$; the backward pass applies the standard matrix-calculus identities $\partial L / \partial W = \text{grad}^\top x$ and $\partial L / \partial x = \text{grad} W$. The helper *transp* witnesses the transposition at the type level by permuting the first two index positions.

Bias addition is a separate *ParaLens'* that broadcasts a vector across the batch dimension and collapses it back on the reverse pass:

```
addLens :: CanAddLens dv dt
      => ParaLens' (Tensor dv dt shape) (Tensor dv dt (b : shape)) (Tensor dv dt (b : shape))
```

```

addLens = lens (uncurry T.add) rev
where rev _ = T.sumDim @ 0 &&& id

```

The gradient with respect to the bias is a sum over the batch axis (@0 passes the axis index to *sumDim*); the gradient with respect to the input is the identity. Composing the two with $\bullet$ yields a full affine layer in a single line:

```

matMulLensCore :: CanMMLens dv dt
  ⇒ ParaLens' (Tensor dv dt [o, i], Tensor dv dt [o]) (Tensor dv dt [b, i]) (Tensor dv dt [b, o])
matMulLensCore = linear • addLens

```

The combined parameter $(Tensor [o, i], Tensor [o])$ arises automatically from the composition rule: the product parameter type requires no manual construction. The full backward pass, computing both $\partial L / \partial W$, $\partial L / \partial b$, and $\partial L / \partial x$, is wired together from the two constituent lenses without any additional code.

Throughout the case studies a convenience alias *matMulLens* is used, which pre-applies gradient descent to both the weight matrix and bias via *repara*:

```

matMulLens :: (CanMMLens dv dt, T.KnownDevice dv)
  ⇒ ParaLens' (MMP dv dt o i) (Tensor dv dt [b, i]) (Tensor dv dt [b, o])
matMulLens = repara (alongside gradUpdate gradUpdate) matMulLensCore

```

Here $MMP\ dv\ dt\ o\ i = (Tensor\ dv\ dt\ [o, i], Tensor\ dv\ dt\ [o])$ is the weight-bias pair type. Using *matMulLens* in a model definition is equivalent to composing *matMulLensCore* with *withGradDesc* after the fact; the difference is purely notational convenience. Chapter 6 uses *matMulLensCore* explicitly to show how any optimiser can be substituted via *repara*.

4.2 Activation Functions

Activation functions carry no learnable parameters and are plain *Lens'* values:

```

sigmoid :: T.StandardFloatingPointDTypeValidation dv dt
  ⇒ Lens' (Tensor dv dt shape) (Tensor dv dt shape)
sigmoid = lens T.sigmoid rev
where rev = (*) ∘ ap (*) (1−) ∘ T.sigmoid
relu    :: T.StandardFloatingPointDTypeValidation dv dt
  ⇒ Lens' (Tensor dv dt shape) (Tensor dv dt shape)
relu = lens T.relu ((*) ∘ heaviside)

```

The sigmoid backward pass uses $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, written point-free as $(*) \circ ap\ (*)\ (1 -) \circ T.sigmoid$: the incoming gradient is multiplied element-wise by $\sigma(x)$ and

$(1 - \sigma(x))$. The ReLU backward multiplies by the Heaviside step $H(x) = \mathbf{1}_{x>0}$, realised as a Boolean tensor cast to the output dtype:

$$\text{heaviside} = T.\text{toDType} @ dt @ T.\text{Bool} \circ \text{liftA2 } (\$) T.\text{gt } T.\text{zerosLike}$$

The $@dt$ and $@T.\text{Bool}$ type applications supply the target and source dtype to toDType : the Boolean comparison mask produced by $T.\text{gt}$ is cast to the floating-point type dt that the rest of the pipeline uses.

Because these layers are plain Lens' values they carry no vertical parameter wires in the wire diagram sense. A Lens' can be appended to a $\text{ParaLens}'$ pipeline using ordinary Haskell function composition ($\circ$) rather than the parametric ($\bullet$). Since $\text{ParaLens}' p a b = \text{Lens}' (p, a) b$, the output type of any $f :: \text{ParaLens}' p a b$ is exactly the input type expected by any $g :: \text{Lens}' b c$, so $f \circ g$ type-checks directly as a $\text{ParaLens}' p a c$ with the parameter type p unchanged:

$$\text{affineRelu} = \text{matMulLensCore} \circ \text{relu}$$

This has the same parameter type $(\text{Tensor } [o, i], \text{Tensor } [o])$ as matMulLensCore alone. The alternative $\text{matMulLensCore} \bullet \text{toPara relu}$ would also type-check but would introduce a spurious unit into the combined parameter: $(\text{Tensor } [o, i], \text{Tensor } [o], ())$.

The operator precedence of ($\bullet$) is set one lower than that of ($\circ$) by design, so multi-layer stacks read cleanly without parentheses:

$$\text{net} = \text{matMulLensCore} \circ \text{relu} \bullet \text{matMulLensCore} \circ \text{relu} \bullet \text{matMulLensCore}$$

This parses as $(\text{matMulLensCore} \circ \text{relu}) \bullet (\text{matMulLensCore} \circ \text{relu}) \bullet \text{matMulLensCore}$: each ($\bullet$) joins two already-clean $\text{ParaLens}'$ values, so the combined parameter is a nested product of weight and bias tensors only, with no unit components anywhere in the type.

4.3 Convolution

Convolution illustrates how `DataKinds`-indexed shapes extend the compile-time safety guarantee beyond simple matrix sizes: the kernel dimensions, input spatial size, stride, and padding must all be consistent, and any mismatch is a type error.

The two-dimensional convolution layer is a $\text{ParaLens}'$ with the convolutional kernel as its parameter:

$$\begin{aligned} \text{convLens} :: & (\text{ConvSideCheck } h \text{ } kH \text{ } 1 \text{ } 0 \text{ } oH, \text{ConvSideCheck } w \text{ } kW \text{ } 1 \text{ } 0 \text{ } oW \\ & , T.\text{All KnownNat } [inC, outC, h, w, b, oH, oW], T.\text{KnownDType } dt, T.\text{KnownDevice } dv) \\ & \Rightarrow \text{ParaLens}' (\text{Tensor } dv \text{ } dt [outC, inC, kH, kW]) \end{aligned}$$

$(Tensor\ dv\ dt\ [b, inC, h, w])$
 $(Tensor\ dv\ dt\ [b, outC, oH, oW])$

The constraints *ConvSideCheck h kH 1 0 oH* and *ConvSideCheck w kW 1 0 oW* encode the output-dimension equations

$$oH = h - k_H + 1, \quad oW = w - k_W + 1$$

for unit stride and zero padding at the type level. A kernel or input of the wrong spatial size is a compile-time type error rather than a runtime failure.

The backward pass is split into two computations. The gradient with respect to the input is obtained via the transposed convolution, the adjoint of the forward operator:

$$\frac{\partial L}{\partial x} = convTranspose2d(kernel, grad)$$

Both *convTranspose2d* and the *im2col* function used below required bug fixes to the *hasktorch* typed API before they could be used here; the details are in Appendix A. The function *im2col* unfolds each spatial patch of the input into a column, producing a tensor of shape $[batch, inC * kH * kW, oH * oW]$. The kernel gradient is then a batched matrix multiply, summed over the batch dimension:

$$\frac{\partial L}{\partial W} = \sum_{batch} grad_{flat} \cdot x_{col}^\top \in \mathbb{R}^{outC \times inC \cdot k_H \cdot k_W}$$

which is reshaped to $[outC, inC, kH, kW]$. This makes the gradient derivation explicit and keeps the backward pass within the verifiable part of the typed API.

4.4 Pooling and Shape

Max pooling carries no parameters and is therefore a plain *Lens'*, with kernel size, stride, and padding all verified by *ConvSideCheck* at the type level:

$maxPool :: (ConvSideCheck\ h\ (Fst\ kernelSize)\ (Fst\ stride)\ (Fst\ padding)\ oH$
 $\quad ,\ ConvSideCheck\ w\ (Snd\ kernelSize)\ (Snd\ stride)\ (Snd\ padding)\ oW)$
 $\Rightarrow Lens'\ (Tensor\ device\ dtype\ [batch, channels, h, w])$
 $\quad (Tensor\ device\ dtype\ [batch, channels, oH, oW])$

The backward pass uses a max-routing mask: the pooled output is expanded back to the input's spatial resolution via *repeat_interleave*, compared element-wise with the original input to identify which positions achieved the maximum, and the incoming gradient flows only through those positions.

The *flatten* lens collapses all non-batch dimensions into a single axis:

$flatten :: KnownNat\ (T.Numel\ shape)$
 $\Rightarrow Lens'\ (Tensor\ dev\ dt\ (batch : shape))$

(*Tensor dev dt [batch, T.Numel shape]*)

The type-level computation *T.Numel shape* computes the total number of elements in the shape list at compile time, so the output width is a compile-time constant. The backward pass is a plain reshape with no gradient arithmetic. Like the activation functions, *maxPool* and *flatten* are plain *Lens'* values and compose into parametric pipelines with $(\circ)$ rather than $(\bullet)$, contributing no parameter wires to the composed diagram.

4.5 Skip Connections

A skip connection adds the input of a sub-network directly to its output, forming a residual shortcut around the learned transformation. In the parametric lens framework this is a *higher-order* combinator: given any endomorphic layer $f :: \text{ParaLens}' \text{ p a a}$, the wrapped layer computes $y = f(x) + x$ and the combined parameter type is unchanged. Gradients flow back through two paths simultaneously (one through f , one through the identity) and are summed at the input:

$$\frac{\partial L}{\partial x} = \underbrace{\frac{\partial L}{\partial y} \cdot f'(x)}_{\text{through } f} + \underbrace{\frac{\partial L}{\partial y}}_{\text{skip}}$$

The skip path ensures that gradients reach early layers even when the learned transformation f saturates or vanishes.

The implementation assembles this from two existing isomorphisms. *splitIso :: Iso a a (a, a) (a, a)* duplicates its argument in the forward direction and sums the two results in the backward direction; *from splitIso* is the reverse: it adds in the forward direction and duplicates in the backward. (*from :: AnIso s t a b → Iso b a t s* inverts an isomorphism, swapping its two directions [9].) Together with *rightLens*, *rotate*, and *alongside*, four combinators suffice:

skipPara :: Num a ⇒ ParaLens' p a a → ParaLens' p a a

skipPara f = rightLens splitIso ∘ from rotate ∘ alongside f id ∘ from splitIso

Reading the chain left to right: *rightLens splitIso* fans the data wire a to (a, a) while leaving the parameter wire p intact; *from rotate* rearranges $(p, (a, a))$ into $((p, a), a)$ so that *alongside* can feed one copy to f , together with its parameters, and the other to *id*; the focus of *alongside f id* is the pair $(f(x), x)$; *from splitIso* sums the pair to produce $f(x) + x$ in the forward direction.

The backward pass is the exact mirror of the forward pass, and no additional gradient logic is needed. *from splitIso*'s *setter* distributes $\partial L / \partial y$ to both branches, precisely because its *getter* summed them in the forward direction. Symmetrically, *rightLens splitIso*'s *setter*

sums the two arriving input gradients into $\partial L/\partial x$, because its *getter* fanned x to two copies going forward.

The elegance is that *splitIso* and *from splitIso* are a single isomorphism and its inverse. Forward, *splitIso* *splits* the input into two copies and *from splitIso* *joins* them by summation; backward, the roles swap. This is automatic: a lens encodes its forward map in the getter and its CRDC inverse in the setter, so a split one way is a join the other. The residual formula $\partial L/\partial x = \partial L/\partial x|_f + \partial L/\partial y$ thus emerges for free, written nowhere in the implementation.

A two-layer residual block is a single application of *skipPara*:

$$resBlock = skipPara (matMulLens \circ relu \bullet matMulLens \circ relu)$$

The parameter type $ResBlockP \text{ dev } dt = (MMP \text{ dev } dt \text{ Hidden Hidden}, MMP \text{ dev } dt \text{ Hidden Hidden})$ holds the two weight-bias pairs, inherited without modification from the inner pipeline.

Projection Skips

skipPara requires its sub-network to be *type-preserving*: $f :: ParaLens' p \ a \ a$. Real ResNet stages that change spatial resolution or channel count cannot satisfy this; the original paper handles such transitions with a *projection shortcut* $y = f(x) + P(x)$, where P matches the output dimension. The natural generalisation runs f and a learned projection *proj* in parallel over a shared input and sums:

$$\begin{aligned} projSkipPara &:: (Num \ b, Num \ b') \\ &\Rightarrow ParaLens \ p \ p' \ a \ a' \ b \ b' \rightarrow ParaLens \ q \ q' \ a \ a' \ b \ b' \rightarrow ParaLens \ (p, q) \ (p', q') \ a \ a' \ b \ b' \\ projSkipPara \ f \ proj &= rightLens \ splitIso \circ r \circ alongside \ f \ proj \circ from \ splitIso \end{aligned}$$

The structure mirrors *skipPara*: *from splitIso* fans the input a to two copies; *alongside f proj* runs both branches in parallel, each with its own independent parameter wire; *rightLens splitIso* sums the two outputs. The auxiliary r is a bookkeeping isomorphism that rearranges the nested parameter–data pairs so that *alongside* receives the correct shape; it carries no gradient logic and is compiled away entirely.

The same *splitIso/from splitIso* duality applies: the gradient formula $\partial L/\partial x = \partial L/\partial x|_f + \partial L/\partial x|_P$ falls out automatically, for the same reason as in *skipPara*. Passing *toPara id* for *proj* recovers the identity-shortcut case: the skip path contributes no learnable parameters and the combined type simplifies accordingly.

4.6 Attention

Self-attention is the most structurally complex layer in this chapter: each output position is a weighted average of all positions, with weights that are themselves a differentiable

function of the input. The layer is *stateful*, its learnable parameters being four square projection matrices, and it fits the *ParaLens'* abstraction unchanged. The parameter type groups them in a tuple:

```
type SelfAttnP dev dt e =
  (Tensor dev dt [e, e] -- Wq
  , Tensor dev dt [e, e] -- Wk
  , Tensor dev dt [e, e] -- Wv
  , Tensor dev dt [e, e] -- Wo
  )
```

With batch size b , sequence length s , and embedding dimension e fixed at the type level, the signature of *selfAttention* is:

```
selfAttention :: (T.All KnownNat [b, s, e]
  , T.MatMulDTypeIsValid dev dt
  , T.BasicArithmeticDTypeIsValid dev dt
  , T.StandardFloatingPointDTypeValidation dev dt
  , T.SumDType dt ~ dt, T.SumDTypeIsValid dev dt
  , KnownNat (b * s) -- Implied
  , (b * (s * e)) ~ ((b * s) * e) -- Implied
  => ParaLens' (SelfAttnP dev dt e) (Tensor dev dt [b, s, e]) (Tensor dev dt [b, s, e])
```

Forward pass. Given input $X \in \mathbb{R}^{b \times s \times e}$, scaled dot-product attention projects X into query, key, and value spaces, scores them, normalises by softmax, and averages the values:

$$Q = XW_Q^\top, \quad K = XW_K^\top, \quad V = XW_V^\top, \quad S = \frac{1}{\sqrt{e}} QK^\top, \quad A = \text{softmax}(S), \quad \text{out} = AVW_O^\top$$

All six intermediate values are produced by a helper *runFwd* used by both the getter and the setter. The getter returns the final projection; the setter calls *runFwd* a second time to reconstruct the intermediates, since the lens carries no channel for cached activations (Section 8.3):

```
runFwd (wq, wk, wv, wo) x =
  let q    = proj wq x
      k    = proj wk x
      v    = proj wv x
      scores = T.mulScalar scale $ T.matmul q $ tr k
      weights = T.softmax @ 2 scores
      attn  = T.matmul weights v
  in (q, k, v, weights, attn, proj wo attn)
```

where $\text{proj } w \ x = T.\text{matmul } x \ (T.\text{transpose} @ 0 @ 1 \ w)$ computes $xW^\top$, $\text{tr} = T.\text{transpose} @ 1 @ 2$ transposes the last two index positions, and scale is the compile-time constant $1/\sqrt{e}$.

Backward pass. The setter applies the chain rule in the reverse order of the graph. At each node the standard matrix-calculus identity for $Y = XA^\top$, namely $\partial L/\partial X = (\partial L/\partial Y) A$ and $\partial L/\partial A = (\partial L/\partial Y)^\top X$, is applied.

Output projection. Differentiating $\text{out} = \text{attn } W_O^\top$:

$$\frac{\partial L}{\partial \text{attn}} = \frac{\partial L}{\partial \text{out}} W_O, \quad \frac{\partial L}{\partial W_O} = \left(\frac{\partial L}{\partial \text{out}} \right)^\top \text{attn}$$

Both tensors are reshaped to $[b \cdot s, e]$ before the matrix multiply, yielding $\partial L/\partial W_O \in \mathbb{R}^{e \times e}$. The helper $w\text{Grad}$ encapsulates this flatten-matmul pattern and is reused identically for all four projection matrices, playing the same role that $T.\text{matmul } (\text{transp grad}) \ x$ plays in *linear*:

$$w\text{Grad } x \ dY = T.\text{matmul } (T.\text{transpose} @ 0 @ 1 \ (T.\text{reshape} @ [b * s, e] \ dY)) \ (T.\text{reshape} @ [b * s, e] \ x)$$

Weighted sum. Differentiating $\text{attn} = AV$:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \text{attn}} V^\top, \quad \frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial \text{attn}}$$

Softmax. The Jacobian of softmax contracts to a rank-one correction. For a single row $y = \text{softmax}(x)$ and incoming gradient g :

$$\frac{\partial L}{\partial x_i} = y_i \left(g_i - \sum_j y_j g_j \right)$$

The inner dot product is computed with $\text{sumDim} @ 2$ along the key axis, reshaped to $[b, s, 1]$ for broadcasting, and the result is scaled element-wise by A :

$$\text{softmaxBwd } w \ dw = w * T.\text{sub } dw \$ T.\text{reshape} @ [b, s, 1] \$ T.\text{sumDim} @ 2 \ (w * dw)$$

Scaled dot-product. Differentiating $S = \frac{1}{\sqrt{e}} QK^\top$:

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{e}} \frac{\partial L}{\partial S} K, \quad \frac{\partial L}{\partial K} = \frac{1}{\sqrt{e}} \left(\frac{\partial L}{\partial S} \right)^\top Q$$

Input projections. $w\text{Grad}$ is applied once per projection to obtain $\partial L/\partial W_Q$, $\partial L/\partial W_K$, and $\partial L/\partial W_V$. The gradient with respect to the input X accumulates the three back-projections:

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Q} W_Q + \frac{\partial L}{\partial K} W_K + \frac{\partial L}{\partial V} W_V$$

The complete setter, with $\text{mm} = T.\text{matmul}$, $\text{tr} = T.\text{transpose} @ 1 @ 2$, and scale' denoting multiplication by $1/\sqrt{e}$, is:

$$\text{rev } (p @ (wq, wk, wv, wo), x) \text{ } dOut = ((dWq, dWk, dWv, dWo), dX)$$

where

$$\begin{aligned} (q, k, v, weights, attn, _) &= \text{runFwd } p \ x \\ dAttn &= T.\text{matmul } dOut \ wo \\ dWo &= wGrad \ attn \ dOut \\ dWeights &= mm \ dAttn \ (tr \ v) \\ dV &= mm \ (tr \ weights) \ dAttn \\ dScores &= \text{softmaxBwd } weights \ dWeights \\ (dQ, dK) &= (scale' \$ mm \ dScores \ k, scale' \$ mm \ (tr \ dScores) \ q) \\ (dWq, dWk, dWv) &= (wGrad \ x \ dQ, wGrad \ x \ dK, wGrad \ x \ dV) \\ dX &= T.\text{matmul } dQ \ wq + T.\text{matmul } dK \ wk + T.\text{matmul } dV \ wv \end{aligned}$$

Multi-head attention. *multiHeadSelfAttention* partitions the embedding dimension across h independent attention heads, each of width $hd = e/h$, enforced at the type level by $e \sim h * hd$. The parameter type is unchanged (the weight matrices remain $[e, e]$ and operate on the full embedding), but two reshape helpers split and merge the head dimension around the attention computation:

$$\begin{aligned} \text{splitHeads} &= T.\text{transpose} @ 1 @ 2 \circ T.\text{reshape} @ [b, s, h, hd] \\ \text{mergeHeads} &= T.\text{reshape} @ [b, s, e] \circ T.\text{transpose} @ 1 @ 2 \end{aligned}$$

splitHeads maps $[b, s, e] \rightarrow [b, s, h, hd] \rightarrow [b, h, s, hd]$: the *reshape* partitions each embedding vector into h slices of width hd , and the *transpose* brings the head axis next to the batch axis, so batched matrix multiplies treat $b \times h$ as one leading dimension. The constraint $\text{Numel } [b, s, e] \sim \text{Numel } [b, s, h, hd]$ witnesses that the reshape preserves element count.

Two details differ from the single-head case. First, the scale factor is $1/\sqrt{hd}$ rather than $1/\sqrt{e}$: within each head the key–query dot products grow with the per-head width hd , not with the full embedding e . Second, softmax is applied along dimension 3 (the key-position axis in the layout $[b, h, s, s]$) rather than dimension 2, and the softmax backward uses *sumDim* @ 3 and a broadcast reshape to $[b, h, s, 1]$ accordingly.

The backward pass is structurally identical. Gradients for Q , K , and V are computed in the $[b, h, s, hd]$ layout and collapsed to $[b, s, e]$ via *mergeHeads* before *wGrad*; since the projection matrices stay $[e, e]$ regardless of h , *wGrad* is shared unchanged.

Both attention variants are *ParaLens'* values: their 4-tuple parameter type is absorbed into the enclosing model's product type by $(\bullet)$ with no extra wiring, exactly as for every other layer in this chapter.

Chapter 5

Loss Functions

A neural network assembled from *ParaLens'* values computes a typed transformation from inputs to predictions, but it does not yet know *what* to optimise. Loss functions supply this missing piece. In the lens framework they are not a separate concept bolted on from the outside; they are simply further *ParaLens'* values that can be composed with the network in the ordinary way. This chapter describes their type, their role in the composed diagram, and the three concrete instances provided by the library.

5.1 Losses, Caps, and the Learning Rate

Every loss function in this framework has the type

$$ParaLens' (t \text{ shape}) (t \text{ shape}) (t [])$$

Reading the three type arguments: the *parameter* $p = t \text{ shape}$ is the target tensor; the *input* $a = t \text{ shape}$ is the network's prediction; and the *output* $b = t []$ is a scalar: a zero-dimensional tensor carrying a single floating-point number.

The loss reduces the network's prediction to a scalar, but it does not yet close the diagram to the monoidal unit. Composing a network *net* with a loss *loss* via $(\bullet)$ gives a term whose output is still $t []$, not the unit type:

$$net \bullet loss : ParaLens' (net_params, t \text{ shape}) (t \text{ input_shape}) (t []).$$

The true *cap*, the morphism that closes the final wire to the monoidal unit $I = ()$, is the learning rate. The library formalises this as a type synonym:

$$\mathbf{type} \ LRLens \ l \ l' = Lens \ l \ l' \ () \ ()$$

LRLens $l \ l'$ has $()$ as *both* output types: the forward pass discards the scalar (outputting the unit), and the backward pass emits the learning rate. The general constructor is:

$$\begin{aligned} learningRate &:: (l \rightarrow l') \rightarrow LRLens \ l \ l' \\ learningRate \ alpha &= lens \ (const \ ()) \ (const \circ \ alpha) \end{aligned}$$

where $const \circ alpha$ ignores the upstream $()$ and applies $alpha$ to the current state l to produce the gradient seed l' . For a constant tensor learning rate, $lrSmoothT$ specialises this by wrapping a Haskell scalar in a rank-0 tensor and negating it:

$$lrSmoothT\ lr = learningRate\ (const\ (negate\ (scalarT\ lr)))$$

Composing with $(\circ)$ closes the diagram fully:

$$\begin{aligned} & (withGradDesc\ net \bullet loss) \circ lrSmoothT\ lr \\ & :: ParaLens'\ (net_params, target)\ input\ () \end{aligned}$$

The output type is now $()$, the monoidal unit, so no information leaves the diagram in the forward direction. The backward pass flows $-lr$ back through every layer as the gradient seed, updating all weight tensors in a single traversal. The *withGradDesc* wrapper (Chapter 6) handles the CRDC natural addition $p \leftarrow p + \partial p$; the cap supplies the sign and magnitude.

5.2 The Gradient Scaling Convention

As established in Section 2.3, the training loop performs the CRDC natural addition

$$\theta_{\text{new}} = \theta_{\text{old}} + \partial\theta,$$

where $\partial\theta$ is emitted by the backward pass; its sign and magnitude are the sole determinants of whether the step is descent or ascent. The backward pass of the MSE loss (Section 5.3) produces

$$\partial p = \frac{2}{N} \alpha \cdot (p - t),$$

where t is the target, N is the total number of output elements, and $alpha$ is the scalar b' received from above. With $\alpha = -\eta$ (a *negative* learning rate), the update becomes

$$p_{\text{new}} = p_{\text{old}} + \frac{2}{N}(-\eta)(p - t) = p_{\text{old}} - \frac{2\eta}{N}(p - t),$$

which is exact gradient descent on the mean-squared-error loss. The $\frac{2}{N}$ factor arises from differentiating $\frac{1}{N} \sum_i (p_i - t_i)^2$; without it the update would correspond to a different (unnormalsed) loss, breaking equivalence with standard automatic differentiation frameworks. The negative sign is not a convention trick; it is the mechanism by which the additive CRDC structure produces descent rather than ascent. Choosing $\alpha > 0$ gives ascent; the library includes *deepDreamLoss* (a dot-product loss $\langle t, p \rangle$ with symmetric gradients) to support activation-maximisation settings without any change to the training loop.

5.3 Mean-Squared Error: `lossSmooth`

The MSE loss is a *ParaLens'* over tensors of arbitrary *shape*:

```

lossSmooth
  :: ( TrivialFacts shape
      , T.BasicArithmeticDTypeIsValid dv dt
      , T.StandardFloatingPointDTypeValidation dv dt )
  => ParaLens' (t shape) (t shape) (t [])

```

The constraint alias *TrivialFacts shape* captures two hasktorch API requirements

```

type TrivialFacts shape =
  (shape ~ T.Broadcast shape shape
   , shape ~ T.Reverse (T.Reverse shape))

```

—that are semantically trivial (every shape satisfies them) but must be supplied as explicit witnesses to satisfy the type-checker when calling *T.sub* and *T.mul*.

The implementation is:

```

lossSmooth = lens fwd (flip rev')
where
  fwd      = uncurry $ T.mseLoss @ T.ReduceMean
  rev' alpha = (id &&& T.neg) o T.mul alpha o uncurry T.sub
              -- fans out gradient to both outputs: (d, -d)

```

The forward pass delegates to *T.mseLoss @ T.ReduceMean*, which computes $\frac{1}{n} \sum_i (t_i - p_i)^2$ averaged over all elements. The backward pass applies the chain rule: with N elements and incoming scalar α from the learning-rate cap,

$$\partial t = \frac{2}{N} \alpha (t - p), \quad \partial p = -\frac{2}{N} \alpha (t - p).$$

5.4 Softmax Cross-Entropy: `softMaxCELoss`

For classification tasks the cross-entropy loss fuses the softmax into the loss for numerical stability:

```

softMaxCELoss
  :: ( T.All KnownNat [x, c]
      , T.BasicArithmeticDTypeIsValid dv dt
      , T.StandardFloatingPointDTypeValidation dv dt
      , T.SumDTypeIsValid dv dt, T.MeanDTypeValidation dv dt )
  => ParaLens' (t [x, c]) (t [x, c]) (t [])

```

The parameter is a distribution over c classes for each of x examples (one-hot or soft labels); the input is the corresponding logit tensor.

softMaxCELoss = *lens fwd rev*

where

fwd (*bt*, *bp*) = *negate* $\circ$ *T.meanAll* $\circ$ *T.sumDim* @ 1

$\$$ *bt* * *T.logSoftmax* @ 1 *bp*

rev (*bt*, *bp*) *d* = (*T.mul* *d'* $\$$ *negate* $\$$ *T.log* *q*, *T.mul* *d'* $\$$ *q* - *bt*)

where

q = *T.softmax* @ 1 *bp*

d' = *T.mulScalar* (*recip* (*fromIntegral* (*T.natValI* @ *x*)) :: *Float*) *d*

The forward pass computes

$$-\frac{1}{x} \sum_{i=1}^x \sum_{c=1}^C t_{ic} \log q_{ic}, \quad q = \text{softmax}(p),$$

the standard categorical cross-entropy averaged over the batch. Throughout, the @1 type application selects axis 1 (class dimension) so that softmax and the summation operate over classes while leaving axis 0 (the batch) intact.

The backward pass exploits the well-known simplification that arises when softmax and cross-entropy are fused. Differentiating the mean over x examples gives a $\frac{1}{x}$ factor that must be carried through: the gradient with respect to the logits is

$$\frac{d}{x} (q - t),$$

where d is the upstream scalar and x is the batch size known at compile time via the type-level natural *T.natValI* @ x . Without this factor the effective learning rate would scale with the batch size, breaking equivalence with standard automatic-differentiation frameworks. The gradient with respect to the targets, $-\frac{d}{x} \log q$, is available because the lens type treats both arguments symmetrically; in practice the training loop discards it, but it is useful in meta-learning settings where the targets themselves are learnable.

Chapter 6

Optimisers

A layer such as *matMulLensCore* knows how to compute gradients with respect to its weights but not what to *do* with them. The gradient update rule (gradient descent, momentum, Adam) is a separate concern, and the framework keeps it separate. Optimisers are lenses plugged into the parameter slot of a layer via *repara* (Section 3.7), converting raw gradients into updated parameters without touching the layer’s forward or backward pass.

6.1 Gradient Descent

The simplest update rule is plain gradient descent: add the incoming gradient directly to the current parameter. In a cartesian reverse differential category this is the *natural addition* $\theta \leftarrow \theta + \partial\theta$; the sign of $\partial\theta$ (controlled by the learning-rate cap) determines whether the step is descent or ascent. The implementation is a single line:

$$\begin{aligned} \text{gradUpdate} &:: \text{Num } p \Rightarrow \text{Lens}' p p \\ \text{gradUpdate} &= \text{lens } id (+) \end{aligned}$$

The getter *id* reads the parameter unchanged; the setter (+) adds the incoming gradient p' to the stored parameter p , returning $p + p'$. Equipping any layer with this rule is a one-liner:

$$\begin{aligned} \text{withGradDesc} &:: \text{Num } p \Rightarrow \text{ParaLens } p p a a' b b' \rightarrow \text{ParaLens } p p a a' b b' \\ \text{withGradDesc} &= \text{repara gradUpdate} \end{aligned}$$

The full gradient-descent training step is then:

$$\begin{aligned} &(\text{withGradDesc } \text{net} \bullet \text{loss}) \circ \text{lrSmoothT } lr \\ &:: \text{ParaLens}' (\text{net_params}, \text{target}) \text{ input } () \end{aligned}$$

withGradDesc ensures each backward pass computes $\theta \leftarrow \theta + \partial\theta$; *lrSmoothT* *lr* (Section 5.1) provides the negative seed $\partial\theta = -\eta \cdot \partial L / \partial \theta$ that makes the step descent. Neither component knows about the other.

6.2 Momentum

Gradient descent converges slowly in the presence of high curvature. Momentum methods accumulate a velocity term that damps oscillations and accelerates convergence along low-curvature directions. In the lens framework, a momentum optimiser is a *Lens'* that *expands* the parameter type from *t shape* to *(t shape, t shape)*, carrying the velocity buffer alongside the weights:

type *Momentum* = *Lens'* (*t shape*, *t shape*) (*t shape*)

The state (v, p) flows on the vertical parameter wire: p is the weight tensor read by the layer, and v is the velocity buffer memorised between steps.

SGD with Momentum

momentum :: *Scalar* $a \Rightarrow a \rightarrow \text{Momentum}$

momentum = *lens snd* $\circ$ *momrev*

The getter *snd* extracts the current weights p for the layer. The setter *momrev gamma* (v, p) p' computes:

$$v' = -\gamma v + \partial p, \quad p_{\text{new}} = p + v',$$

where ∂p is the incoming gradient. Setting $\partial p = -\eta \partial L / \partial p$ (from *lrSmoothT*) and expanding gives the standard SGD-with-momentum update. The velocity decays by γ each step and accumulates the gradient.

Nesterov Momentum

nesterov :: *Scalar* $a \Rightarrow a \rightarrow \text{Momentum}$

nesterov gamma = *lens* (*uncurry fwd*) (*momrev gamma*)

where *fwd* v $p = p + T.\text{mulScalar gamma } v$

The only difference from plain momentum is the getter: instead of returning p it returns the lookahead position $p + \gamma v$, so the gradient is evaluated at the next predicted iterate. The setter *momrev gamma* is unchanged. Both *momentum* and *nesterov* plug into any layer via *repara* (*momentum* 0.9) *layer*, which expands the parameter type from p to (v, p) with the velocity buffer initialised automatically.

6.3 Adaptive Methods

Adaptive methods maintain per-parameter statistics and scale the effective learning rate accordingly. They are again *Lens'* values that expand the parameter type.

AdaGrad

$adaGrad :: (Scalar\ a, Fractional\ a) \Rightarrow a \rightarrow Momentum$

$adaGrad\ eps = lens\ snd\ rev$

where

$rev\ (g, p)\ p' = (g', p + update * p')$

where

$g' = g + p' * p'$

$update = T.mulScalar\ eps \circ T.reciprocal$

$\circ T.addScalar\ delta\ \$\ T.sqrt\ g'$

The accumulator g stores the running sum of squared gradients. The per-parameter effective learning rate is $\varepsilon/\sqrt{g' + \delta}$, which shrinks automatically for parameters that receive large or frequent updates, and stays large for infrequently-updated parameters. The constant $\delta = 10^{-7}$ prevents division by zero.

Adam

Adam maintains two exponentially decaying moment estimates:

$adam :: (Scalar\ a, Fractional\ a) \Rightarrow a \rightarrow a \rightarrow a \rightarrow Momentum2$

$adam\ beta1\ beta2\ eps = lens\ snd\ rev$

where

$rev\ ((m, v), p)\ p' = ((m', v'), p + T.mulScalar\ eps\ update)$

where

$m' = T.mulScalar\ beta1\ m + T.mulScalar\ (1 - beta1)\ p'$

$v' = T.mulScalar\ beta2\ v + T.mulScalar\ (1 - beta2)\ (p' * p')$

$update = m' / T.addScalar\ delta\ (T.sqrt\ v')$

where $Momentum2 = Lens' ((t\ shape, t\ shape), t\ shape)\ (t\ shape)$ carries state $((m, v), p)$. m is the first moment (exponential moving average of gradients), v is the second moment (exponential moving average of squared gradients), and ε is the step size. The update $m'/\sqrt{v' + \delta}$ normalises the gradient by its recent root-mean-square, reducing the effective learning rate for high-variance parameters.

Bias correction. This implementation omits the bias-correction factors from the original Adam paper [21], using the raw moments m' and v' directly. The effect is negligible once training is established; including correction would require threading a step counter through the optimiser state, adding a type parameter beyond what the thesis examples require; and beyond the original Cruttwell et al. framework [4].

The deepened state type $((m, v), p)$ vs $Momentum$'s (v, p) is automatic: applying

`repara (adam 0.9 0.999 1e-3)` expands the parameter type to $((m, v), weights)$ with no changes to the layer or the rest of the network.

6.4 Per-layer Optimisers

One key payoff of the *repara* mechanism is that different layers can use *different optimisers* with no coordination between them:

```
net = repara (momentum 0.9) matMulLensCore
      • repara (adaGrad 1e-2) matMulLensCore
```

The combined parameter type is $((v, p_1), (g, p_2))$: each layer carries its own independent optimiser state, tracked automatically through the product type from $(\bullet)$. No global optimiser object is needed; the type system enforces that each layer’s update rule stays local.

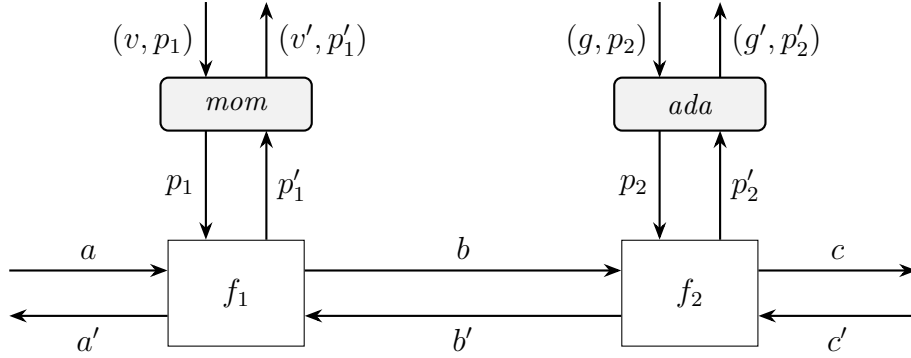


Figure 6.1: Two layers with independent per-layer optimisers. Layer f_1 uses SGD with momentum (state (v, p_1)); layer f_2 uses AdaGrad (state (g, p_2)). Data flows forward on the upper wires ($a \rightarrow b \rightarrow c$) and backward on the lower wires ($c' \rightarrow b' \rightarrow a'$); the two parameter channels are structurally independent, carrying different state types on the vertical axis.

This contrasts with conventional frameworks such as PyTorch or JAX, where the optimiser is a global object that must be explicitly configured with which parameters belong to which update rule, and where mixing rules across layers requires manual parameter grouping. Here the types enforce the separation: the outer parameter type $((v, p_1), (g, p_2))$ is the product of two independent optimiser states, and the composition rule $(\bullet)$ assembles it automatically from the individual *repara* calls.

Chapter 7

Case Studies and Evaluation

The preceding chapters described the framework’s abstractions in isolation. This chapter grounds them in four concrete applications. The first, Iris flower classification, is a standard multi-layer perceptron benchmark small enough to inspect the output of GHC’s optimiser directly. The second, MNIST digit recognition, shows the same compositional style scaling to a convolutional architecture. The third demonstrates that the framework is not limited to discriminative tasks: an MNIST autoencoder is expressed as two ordinary *ParaLens*’ pipelines composed end-to-end, with the bottleneck constraint enforced at the type level. The fourth revisits MNIST with a residual architecture, demonstrating that *skipPara* composes into larger models without modification and that its parameter type is inferred automatically by the type system. Together they demonstrate that the lens abstraction adds zero runtime cost and that the type system catches architectural mistakes at compile time rather than at runtime.

7.1 Iris Flower Classification

The Iris dataset comprises 150 specimens of three species of iris, each described by four real-valued morphological measurements (sepal and petal length and width). The classification task is a standard MLP benchmark and is used here specifically because the network is small enough to read GHC Core output in full and compare it line for line against a hand-written implementation.

Model Definition

The architecture is n hidden fully-connected layers of width 4, followed by a single output layer that maps 4 features to 3 class logits. Three type aliases encode this at the type level:

type *InnerLayer* $dv\ dt = MMP\ dv\ dt\ 4\ 4$

type *IParams* $n\ dv\ dt = (StackedN\ n\ (InnerLayer\ dv\ dt), MMP\ dv\ dt\ 3\ 4)$

$MMP\ dv\ dt\ out\ \mathbf{in} = (T.Tensor\ dv\ dt\ [out, \mathbf{in}], T.Tensor\ dv\ dt\ [out])$ is a weight matrix paired with a bias vector (Chapter 4). *InnerLayer* is a $4 \rightarrow 4$ affine map. *IParams* $n\ dv\ dt$

is the product of n inner layers followed by one output layer; for $n = 2$ it expands to $((MMP\ dv\ dt\ 4\ 4, MMP\ dv\ dt\ 4\ 4), MMP\ dv\ dt\ 3\ 4)$: a fully concrete nested tuple whose structure mirrors the layer order.

The *ParaLens'* model is:

$$\begin{aligned}
nMuls &:: (CanStack\ n, SaneDT\ dv\ dt) \Rightarrow \\
&\quad ParaLens'\ (StackedN\ n\ (MMP\ dv\ dt\ m\ m)) \\
&\quad\quad (T.Tensor\ dv\ dt\ [b, m]) \\
&\quad\quad (T.Tensor\ dv\ dt\ [b, m]) \\
nMuls &= stackN\ @\ n\ (matMulLens\ \circ\ sigmoid) \\
irisModel &:: (CanStack\ n, SaneDT\ dv\ dt) \Rightarrow \\
&\quad ParaLens'\ (Inp\ (T.Tensor\ dv\ dt\ [b, 4]), IParams\ n\ dv\ dt) \\
&\quad\quad () \\
&\quad\quad (Out\ (T.Tensor\ dv\ dt\ [b, 3])) \\
irisModel &= argToPara\ \bullet\ nMuls\ @\ n\ \bullet\ matMulLens\ \circ\ sigmoid
\end{aligned}$$

argToPara (Section 3.7) converts the raw input tensor into a *ParaLens'* with a unit parameter, making it composable with $(\bullet)$. Each $(\bullet)$ join adds one more layer, automatically extending the parameter type. The final *matMulLens* $\circ$ *sigmoid* is the output layer; *relu* is parameter-free so it composes with plain $(\circ)$ rather than with $(\bullet)$, leaving the output layer's parameter type clean.

The hand-rolled forward pass for $n = 1$ is:

$$\begin{aligned}
handRolledTest\ (i, (mb, mb')) &= layer\ mb' \circ layer\ mb\ \$\ i \\
\textbf{where} \\
layer\ (m, b) &= T.sigmoid \circ T.add\ b \circ ('T.matmul' transp\ m)
\end{aligned}$$

The two definitions produce identical GHC Core, as shown in the next subsection.

Zero-Overhead Abstraction

The elimination happens in two stages. First, the van Laarhoven representation

$$Lens\ s\ t\ a\ b = \forall f. Functor\ f \Rightarrow (a \rightarrow f\ b) \rightarrow s \rightarrow f\ t$$

is specialised at the call site. The forward pass (*runFullModel*) instantiates $f = \mathbf{Const}\ b$, for which $fmap = flip\ const$, collapsing every lens in the chain to its getter. The backward pass instantiates $f = \mathbf{Identity}$, for which $fmap = coerce$, collapsing to the setter-getter chain. In both cases the $\forall f$ disappears and all intermediate *fmap* calls and $(,)$ wrappers reduce to their concrete definitions. Second, the `{-# INLINE #-}` pragmas on $(\bullet)$, *repara*, *stackN*, and their auxiliaries (*swapFst*, *rotate*, *leftLens*, *rightLens*) allow GHC to

discharge the remaining composition layers, leaving only the raw tensor operations.

Compiling both `test = runFullModel (irisModel @ 1)` and `handRolledTest` with `-O2` and `-ddump-simpl` produces worker functions `$wtest` and `$whandRolledTest` that are structurally identical.

Each worker accepts nine unboxed arguments (a `ForeignPtr` pair for the input tensor and two pointer pairs per weight/bias group) and performs the same eight operations in the same order:

```

$$wtranspose  w1_ptr          -- W1^T
$wmatmul_tt   x_ptr    W1^T    -- X * W1^T
$wadd         b1_ptr    (X*W1^T) -- + b1
$wsigmoid_t   (X*W1^T + b1)    -- sigma_1
$$wtranspose  w2_ptr          -- W2^T
$wmatmul_tt   h1_ptr    W2^T    -- h1 * W2^T
$wadd         b2_ptr    (h1*W2^T) -- + b2
$wsigmoid     (h1*W2^T + b2)    -- sigma_2

```

The wrapper functions also match: both `test1` and `handRolledTest` perform twelve nested `case` analyses to unpack the input tuple into nine raw pointer arguments, then tail-call their respective workers. The only differences in the entire output are internal variable names and coercion-tag numbers (`Co:42` vs. `Co:960` for the final cast), which are GHC-internal type-representation artefacts with no runtime cost.

The lens machinery—*Lens* (p, a), $(\bullet)$, *alongside*, *swapFst*, *rotate*, the $(,)$ tuple constructors—is entirely absent from the optimised output. The user writes four lines of lens composition; GHC emits the same sequence of LibTorch FFI calls as the hand-written version.

To confirm the property scales with depth, the same `-O2 -ddump-simpl` test was repeated at $n = 20$ and $n = 50$:

```

test' = runFullModel $ irisModel @ 20 -- 1,800 lines of Core
test' = runFullModel $ irisModel @ 50 -- 4,900 lines of Core

```

In both cases the Van Laarhoven $\forall f$, all $(\bullet)$ compositions, *repara*, *swapFst*, *rotate*, and every $(,)$ constructor are absent from the output. Core size grows linearly with n (roughly 103 lines per layer), confirming that *depth* is the controlling variable, not matrix size. The lens abstraction eliminates completely at every tested depth. This suggests the zero-overhead property holds for any practically realisable network; the only theoretical constraint is GHC’s simplifier tick budget (`-fsimpl-tick-factor`), which can be raised

explicitly if a sufficiently deep model ever approaches the default limit (Section 8.3).

Type-Level Layer Depth

The depth n is a compile-time natural number, not a runtime value. `Stack.hs` converts it to a Peano numeral and uses typeclass induction to build both the parameter type and the $(\bullet)$ composition tree simultaneously:

```
data PeanoNat = One | Succ PeanoNat

type family Stacked (n :: PeanoNat) (m :: Type) where
    Stacked One m      = m
    Stacked (Succ n) m = (m, Stacked n m)

class StackN (n :: PeanoNat) where
    stack :: ParaLens' p a a → ParaLens' (Stacked n p) a a

instance StackN One where
    stack l = l

instance StackN n ⇒ StackN (Succ n) where
    stack l = l • stack @ n l
    {-# INLINE stack #-}    -- Crucially important!
```

The public API bridges GHC's built-in *Nat* literals:

```
type StackedN (n :: Nat) m = Stacked (ToPeano n) m
type CanStack n           = StackN (ToPeano n)

stackN :: CanStack n ⇒ ParaLens' p a a → ParaLens' (StackedN n p) a a
stackN = stack @ (ToPeano n)
```

For $n = 3$, `StackedN 3 (MMP...)` resolves at compile time to $(MMP..., (MMP..., MMP...))$. Crucially, GHC always sees a concrete product type, not a list or a vector. This is precisely what enables the zero-overhead property from the previous subsection: because the parameter type is a fully known nested tuple at each n , the inliner can fully specialise and unbox the entire parameter structure, producing the same Core as a hand-written n -layer network at any fixed depth.

Adding or removing a hidden layer is a one-character change to the type application `@n`; no other code changes.

The same typeclass induction extends to initialisation. A *RandInit* instance on pairs lifts random sampling to any product type automatically:

```
instance (RandInit l, RandInit r) ⇒ RandInit (l, r) where
    randInit = liftA2 (,) randInit randInit
```

irisInitParams@n therefore generates a freshly randomised *IParams n dv dt* of the correct depth with no boilerplate.

Devices, Dtypes, and Static Shapes

The tensor type *T.Tensor dv dt shape* carries three compile-time parameters: the device *dv*, the element dtype *dt*, and the shape *shape* (a type-level list of dimension sizes). The *SaneDT* constraint alias bundles the training requirements:

```
type SaneDT dv dt =
  ( T.KnownDType dt
  , T.StandardFloatingPointDTypeValidation dv dt
  , CanMMLens dv dt)
```

T.StandardFloatingPointDTypeValidation dv dt is unsatisfied by integer or boolean dtypes, so any attempt to train with a non-differentiable dtype is rejected by the type checker before a single line of training code runs. Switching between precisions or between CPU and GPU requires only a change to the type application at the call site:

```
irisGetEpoch @ 1 @ (T.CPU, 0) @ T.Double  -- CPU, double precision
irisGetEpoch @ 1 @ (T.CUDA, 0) @ T.Float   -- GPU, single precision
```

The model definition, training loop, and loss function are entirely unchanged. Shape mismatches are type errors caught at compile time. For example, passing a *Tensor* [1, 10] where *Tensor* [1, 4] is expected:

```
type Tensor s = T.Tensor DV DT s
type MMP o i = L.MMP DV DT o i

foo :: ParaLens' (MMP 6 4) (Tensor [b, 4]) (Tensor [b, 6])
foo = matMulLens

x :: Tensor [1, 6]
x = run (p :: MMP 6 4, oops :: Tensor [1, 10])
where run = runModel foo
```

produces:

```
* Couldn't match type '10' with '4'
   Expected: Tensor [1, 4]
   Actual:   Tensor [1, 10]
* In the expression: oops :: Tensor [1, 10]
   In the first argument of 'run', namely
   '(p :: MMP 6 4, oops :: Tensor [1, 10])'
```

In the expression: `run (p :: MMP 6 4, oops :: Tensor [1, 10])`

The mismatch is named directly and no runtime assertion is required. No dynamic shape checks are needed anywhere in the framework.

Training Results

The model was trained for 100 epochs on all 150 Iris examples with batch size 8 and learning rate 10^{-3} .

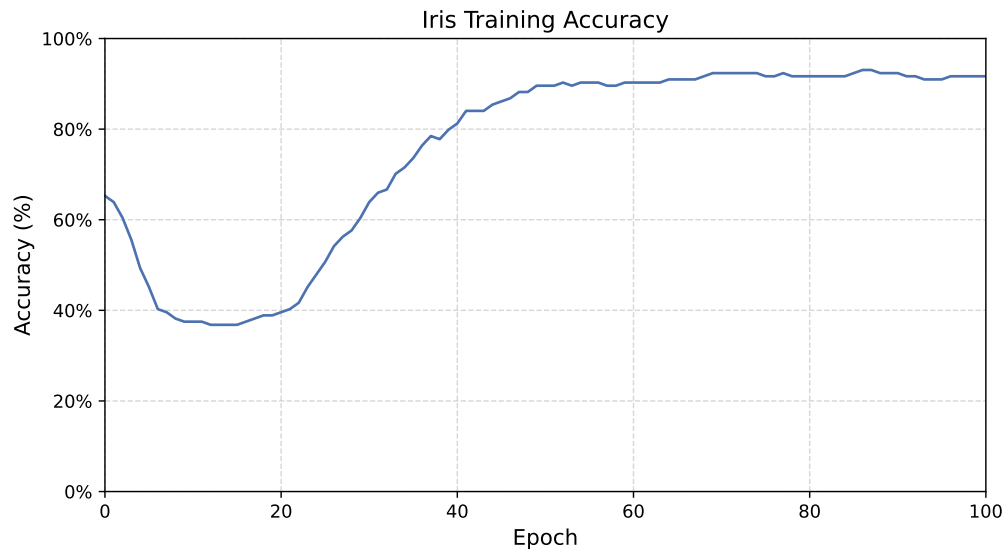


Figure 7.1: Training accuracy on the full 150-example Iris dataset over 100 training epochs. Two hidden layers of width 4 with sigmoid activations, trained with batch size 8 and learning rate 10^{-3} .

Figure 7.1 shows the learning curve. Accuracy rises from 65% at epoch 0 and crosses 90% by epoch 50. The model peaks at **93.06%** at epoch 86, then settles at 91.67% by epoch 100. The small size of the Iris model means that the performance differences between Float and Double are negligible; Table 7.1 in Section 7.2 gives a more realistic benchmark on the larger MNIST dataset.

7.2 MNIST Digit Recognition

MNIST is a dataset of 70,000 greyscale images of handwritten digits (0–9) at 28×28 pixels, split into 60,000 training and 10,000 test examples. The model used here is a small convolutional network followed by a dense output layer:

```
[batch, 1, 28, 28]
conv1 (3x3, 3 filters) -> relu -> maxpool(2x2) -> [batch, 3, 13, 13]
conv2 (4x4, 5 filters) -> relu -> maxpool(2x2) -> [batch, 5, 5, 5]
flatten -> [batch, 125]
```

dense (125 -> 10)

-> [batch, 10]

The 1,527-parameter model (conv1: 27, conv2: 240, dense: 1,260) is intentionally modest; the purpose is to demonstrate the framework’s compositional style on a multi-stage architecture, not to achieve state-of-the-art accuracy.

Performance

Precision	Mean (ms)	Min (ms)	Max (ms)	Std dev (ms)	R^2
<i>T.Float</i> (32-bit)	251	232	260	16	0.993
<i>T.Double</i> (64-bit)	289	284	292	5	1.000

Table 7.1: MNIST CNN training: wall-clock time per epoch by floating-point precision, CPU, batch size 32, 6,000 training examples. Measured with Criterion after a full-epoch warmup pass for each dtype; min/max are the 95% confidence bounds on the mean. Float is 15% faster, consistent with wider SIMD throughput for 32-bit arithmetic.

Parameter Types

Four type aliases encode the architecture at the type level:

type *Conv1K dev dt* = *T.Tensor dev dt* [3, 1, 3, 3]

type *Conv2K dev dt* = *T.Tensor dev dt* [5, 3, 4, 4]

type *DenseP dev dt* = *MMP dev dt* 10 125

type *MnistP dev dt* = (*Conv1K dev dt*, (*Conv2K dev dt*, *DenseP dev dt*))

MnistP is the product of all learnable parameters in layer order. A kernel of the wrong shape: say, [3, 1, 4, 4] instead of [3, 1, 3, 3] for *Conv1K*, is a compile-time type error. The product structure is assembled automatically by ($\bullet$): each composition step extends the parameter tuple by one more layer’s worth of weights, so *MnistP* is inferred rather than written by hand.

The *SaneMnist* constraint alias bundles every capability required for training, including dtype validity for mean reduction (used by the loss) and comparison operations (used by the accuracy metric):

type *SaneMnist dev dt* =
(*CanMMLens dev dt*
, *T.StandardFloatingPointDTypeValidation dev dt*
, *T.MeanDTypeValidation dev dt*
, *T.KnownDevice dev*
, *T.KnownDType dt*)

Any device/dtype combination that fails to satisfy the full set e.g. a device with no mean-reduction support, is rejected at compile time.

Model Composition

mnistModel = *argToPara*

- *withGradDesc convLens* $\circ$ *relu* $\circ$ *maxPool* @ (2, 2) @ (2, 2) @ (0, 0)
- *withGradDesc convLens* $\circ$ *relu* $\circ$ *maxPool* @ (2, 2) @ (2, 2) @ (0, 0)
- *rightLens* (*flatten* @ *b* @ [5, 5, 5]) $\circ$ *matMulLens*

Each (•) arm contributes one block to both the forward pass and the combined parameter type. Within each convolutional arm, *relu* and *maxPool*@... are parameter-free lenses composed with plain ($\circ$) rather than (•), so they thread data through without extending the parameter type. The type applications to *maxPool* (kernel size, stride, and padding) are static; the output shape after each pooling step is checked at compile time. *withGradDesc convLens* (Section 6.1) wraps each convolutional layer with the CRDC natural addition, so the gradient update $\theta \leftarrow \theta + \partial\theta$ is baked into the layer itself.

The final arm uses *rightLens* to pre-process the data component of the (*param*, λ , **data**) pair before the dense layer:

$$(MMP, [batch, 5, 5, 5]) \xrightarrow{\text{rightLens (flatten)}} (MMP, [batch, 125]) \xrightarrow{\text{matMulLens}} [batch, 10].$$

flatten is a pure *Iso'* with no learnable parameters and no effect on the gradient path through *MMP*. Composing it via *rightLens* (which lifts a lens on the second component of a pair) keeps the reshape and the matmul as a single *ParaLens'* unit, avoiding any intermediate allocation.

He Initialisation

mnistInitParams :: *IO* (*MnistP dev dt*)

mnistInitParams = **do**

```

let sc x    =    T.mulScalar (x :: Float)
c1 ← sc (sqrt (2 / 9))    ⟨$⟩ T.randn
c2 ← sc (sqrt (2 / 48))   ⟨$⟩ T.randn
w  ← sc (sqrt (2 / 125))  ⟨$⟩ T.randn
let b = T.zeros
pure (c1, (c2, (w, b)))

```

He initialisation [22] sets each layer's weight standard deviation to $\sqrt{2/\text{fan_in}}$, keeping the variance of activations constant across ReLU layers and preventing vanishing gradients in early training. The fan-in values are $1 \times 3 \times 3 = 9$ for *Conv1K*, $3 \times 4 \times 4 = 48$ for *Conv2K*, and 125 for the dense weight matrix. The bias is initialised to zero.

Loss and Training

The loss and training models follow the same pattern as Iris:

$$\begin{aligned} \text{mnistModelLoss} &= \text{mnistModel} \bullet \text{softMaxCELoss} \\ \text{mnistModel}' &= \text{mnistModel} \bullet \text{softMaxCELoss} \circ \text{lrSmooth } 1\text{e-}4 \end{aligned}$$

softMaxCELoss (Section 5.4) fuses the softmax and log operations for numerical stability. *lrSmooth* $1\text{e-}4$ caps the output wire to the unit type, seeding backpropagation with -10^{-4} .

One training epoch is a strict left fold over all batches:

$$\text{mnistEpoch } \text{trainT } \text{params} = \text{trainMany } \text{mnistModel}' \text{ } \text{params } \text{trainT}$$

Inference extracts predictions by taking the argmax along the class axis:

$$\begin{aligned} \text{mnistPredict } p \ x = \\ &U.\text{asValue} \circ T.\text{toDynamic } \$ \\ &T.\text{argmax } @1 @ T.\text{DropDim } (\text{runFullModel } \text{mnistModel } (x, p)) \end{aligned}$$

The `@1` selects the class dimension and `@T.DropDim` removes it, returning a list of integer digit labels. The type of *runFullModel* *mnistModel* (x, p) is *T.Tensor dev dt [b, 10]*, so the argmax and the label extraction are statically typed throughout.

Training Results

The model was trained for 208 epochs on a 6,000-example subset of the MNIST training set (10%)—chosen to keep each epoch fast enough to run on a CPU-only machine—with batch size 32 and learning rate 10^{-4} . Test accuracy was evaluated on the full 10,000-example test set after each epoch.

Figure 7.2 shows the learning curve. Accuracy rises steeply in the first 50 epochs as the convolutional filters specialise to digit strokes, then transitions to a slower climb. The model first crosses **90%** at epoch 151 (90.22%) and peaks at **91.43%** at epoch 200, after which it plateaus. Given that the model has only 1,527 parameters and is trained on one-tenth of the available data, this result confirms that the framework’s compositional forward and backward passes are numerically correct end-to-end and that the lens abstraction imposes no obstacle to learning.

7.3 Residual MNIST

The MNIST task is revisited with a residual architecture to show that *skipPara* (Section 4.5) integrates into a larger pipeline without modification. The model replaces the

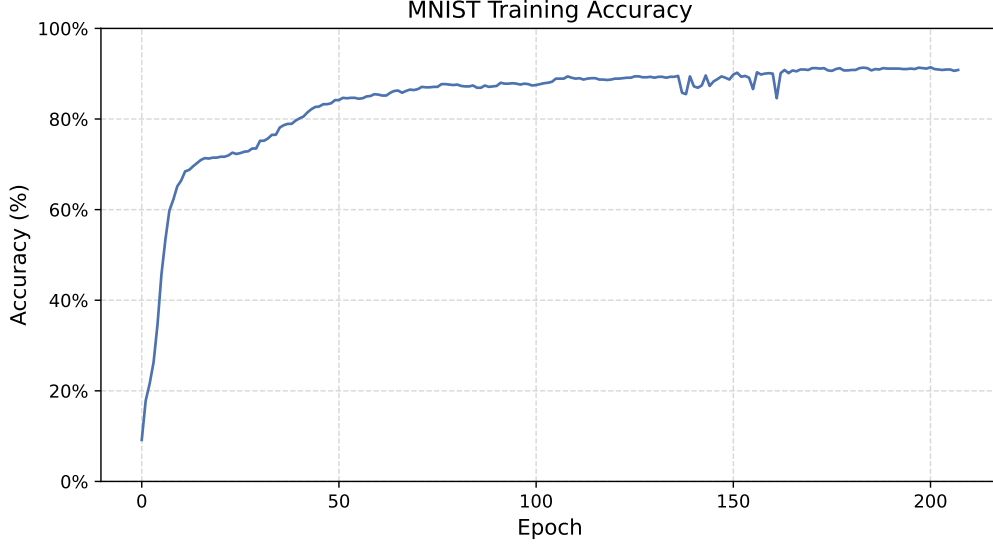


Figure 7.2: Test accuracy on the full 10,000-example MNIST test set over 208 training epochs. The model is a 1,527-parameter CNN trained on 6,000 examples with batch size 32 and learning rate 10^{-4} .

convolutional stack with a sequence of dense residual blocks, keeping the setting comparable to the plain MNIST model.

Parameter Types

Each residual block wraps two affine layers of the same width, so its parameter type is a pair of weight-bias pairs:

type *Hidden* = 128

type *NumBlocks* = 3

type *ResBlockP dev dt* = (*MMP dev dt Hidden Hidden*, *MMP dev dt Hidden Hidden*)

type *ResMnistP dev dt* =

(*MMP dev dt Hidden* 784

, (*StackedN NumBlocks (ResBlockP dev dt)*, *MMP dev dt 10 Hidden*))

ResMnistP is the product of an input projection, three stacked block parameter pairs, and an output layer, assembled automatically by the ($\bullet$) composition rule.

Model Composition

A single residual block is one application of *skipPara*:

resBlock :: *SaneRes b dev dt*

$\Rightarrow$ *ParaLens'* (*ResBlockP dev dt*)

(*Tensor dev dt [b, Hidden]*)

(*Tensor dev dt [b, Hidden]*)

$resBlock = skipPara (matMulLens \circ relu \bullet matMulLens)$

The parameter type ($ResBlockP \ dev \ dt$) is inherited from the inner pipeline; $skipPara$ contributes no additional parameters. The full model stacks $NumBlocks$ such blocks between a flattening input projection and a linear output layer:

$resMnistModel = argToPara$

- $rightLens (flatten \ @ \ b \ @ \ [1, 28, 28]) \circ matMulLens \circ relu$
- $stackN \ @ \ NumBlocks \ resBlock$
- $matMulLens$

The pipeline is a drop-in replacement for $mnistModel$: the same training loop, loss combinator, and inference infrastructure apply unchanged.

He Initialisation

Weights use the same $\sqrt{2/fan_in}$ rule as the MNIST model above, with fan-in 784 for the input projection and $Hidden = 128$ for every layer inside a residual block.

Longer-Range Skips

The architecture above chains three blocks with $stackN$, each carrying an independent $skipPara$ connection spanning its own two layers: the standard residual block structure of He et al. [22]. The blocks are composed sequentially with no cross-block skip connections, matching the original ResNet design. For stages that change spatial resolution or channel count, the framework provides $projSkipPara$ (Section 4.5), which runs a learned projection shortcut in parallel with f using the same $splitIso$ fan-out and sum structure.

Training Results

The residual model was trained for 300 epochs on the same 6,000-example subset of MNIST used for the CNN above, with batch size 32 and learning rate 10^{-4} .

Figure 7.3 shows the learning curve. Starting near random (14.8%), accuracy rises steeply through the first 100 epochs, crossing 70% at epoch 104. Progress slows in the middle phase: 75% is reached at epoch 203, and the model peaks at **78.63%** at epoch 289. The plateau reflects a structural limitation of plain gradient descent: once the majority of training examples are classified confidently, the softmax gradient $\frac{1}{B}(q - t)$ approaches zero for correct predictions and only the remaining incorrect examples contribute signal. The plateau is therefore a property of the optimiser, not of the architecture.

The result demonstrates that $skipPara$ and $stackN$ compose correctly in a multi-block pipeline: gradient flows back through each skip connection without additional wiring, the

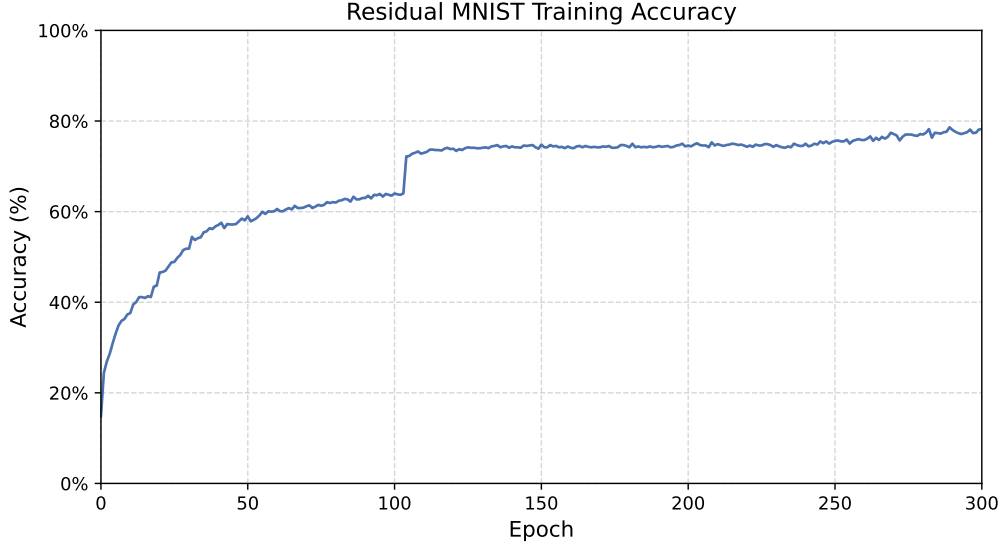


Figure 7.3: Test accuracy of the residual MNIST model over 300 training epochs. Three dense residual blocks of width 128, trained on 6,000 examples with batch size 32 and learning rate 10^{-4} .

combined parameter type *ResMnistP* is assembled and updated automatically, and the model achieves non-trivial accuracy from a clean four-line definition.

7.4 MNIST Autoencoder

Autoencoders demonstrate that the framework is not limited to discriminative tasks. An autoencoder learns to compress its input into a low-dimensional latent representation and then reconstruct the original from that representation, trained by minimising reconstruction error rather than classification loss. In the parametric lens framework, encoder and decoder are each ordinary *ParaLens'* pipelines; composing them end-to-end with $(\bullet)$ produces the full model with no special casing.

Architecture

The model maps flattened MNIST images ($28 \times 28 = 784$ pixels) to a 32-dimensional latent space and back:

```
encoder: [b, 784] -> Dense(784->128) -> ReLU -> Dense(128->32)
decoder: [b, 32] -> Dense(32->128) -> ReLU -> Dense(128->784) -> Sigmoid
```

The bottleneck latent dimension 32 is strictly smaller than input dimension 784, and enforced at the type level: the encoder output type *T.Tensor dv dt [b, LatentDim]* must unify with the decoder input type, so a dimension mismatch is a compile-time error rather than a silent shape broadcast.

Parameter Types

Three type aliases capture the parameter structure:

```

type EncoderP dev dt = (MMP dev dt HiddenDim InputDim,
                          MMP dev dt LatentDim HiddenDim)
type DecoderP dev dt = (MMP dev dt HiddenDim LatentDim,
                          MMP dev dt InputDim HiddenDim)
type AEP dev dt       = (EncoderP dev dt, DecoderP dev dt)

```

where *InputDim* = 784, *HiddenDim* = 128, and *LatentDim* = 32. *AEP dev dt* is a nested product of four weight-bias pairs. As with the discriminative models, this type is not written by hand: it is assembled automatically by $(\bullet)$ from the encoder and decoder parameter types.

Model Composition

Encoder and decoder are each plain *ParaLens'* values:

```

encoderCore :: (SaneAE dev dt, KnownNat b)
              => ParaLens' (EncoderP dev dt)
                  (T.Tensor dev dt [b, InputDim])
                  (T.Tensor dev dt [b, LatentDim])
encoderCore = matMulLens ∘ relu • matMulLens
decoderCore :: (SaneAE dev dt, KnownNat b)
              => ParaLens' (DecoderP dev dt)
                  (T.Tensor dev dt [b, LatentDim])
                  (T.Tensor dev dt [b, InputDim])
decoderCore = matMulLens ∘ relu • matMulLens ∘ sigmoid

```

The full autoencoder is a single further composition:

```

autoencoderModel = argToPara • encoderCore • decoderCore

```

Three $(\bullet)$ calls assemble the complete pipeline. The combined parameter type $(T.Tensor\ dev\ dt\ [b, InputDim], AEP\ dev\ dt)$ is inferred without annotation. The composition rule pairs each sub-lens's parameter wire into the product type automatically, in exactly the same way as the discriminative models of the preceding sections.

Loss and Training

Reconstruction quality is measured by mean-squared error. The trainable model is:

```

autoencoderModel' = autoencoderModel • lossSmooth ∘ lrSmooth 1e-3

```

lossSmooth (Section 5.3) replaces the cross-entropy loss of earlier models; no other part of the training infrastructure changes. Each training step presents a batch x of flattened images paired with itself as the reconstruction target:

aeEpoch trainT = *flip* (*trainMany* (*autoencoderModel'* @ *BatchSize*)) *trainT*

where *trainT* is a list of (x, x) pairs. The parameter update, gradient flow, and epoch structure are handled identically to the discriminative case. The only difference visible at the call site is the loss: *lossSmooth* computes $\frac{1}{N} \sum_i (p_i - t_i)^2$ rather than cross-entropy, and the gradient seed flows backward through the decoder and into the encoder without any additional wiring.

7.5 Performance Evaluation

Three implementations of the Iris two-layer MLP at depth $n = 1$ are compared using Criterion [23] on a CPU-only machine (GHC 9.8.4, LibTorch 2.9.1):

typed-hasktorch The *ParaLens* library with *Torch.Typed* tensors; forward and backward passes are pure Haskell lens composition, calling LibTorch only for primitive tensor operations.

hmatrix A second lens-based implementation wrapping LAPACK and BLAS via the HMatrix library [19]; the backward pass is also pure Haskell.

dynamic-hasktorch A baseline delegating both passes to PyTorch autograd via *runStep*, *flattenParameters*, and *.backward()*.

raw_fwd measures a single forward pass over one batch of eight samples; **epoch** measures one full training epoch, a strict left fold over all 18 mini-batches, including the backward pass and parameter update for every batch.

Benchmark	Implementation	Mean time	Rel. to typed
raw_fwd	typed-hasktorch	6.9 μs	1.00×
	typed-hasktorch-handroll	6.8 μs	0.99×
epoch	hmatrix	390 μs	0.57×
	typed-hasktorch	689 μs	1.00×
	dynamic-hasktorch	3,631 μs	5.27×

Table 7.2: Criterion benchmark results for the Iris MLP ($n = 1$, batch size 8, 18 batches per epoch, CPU). All times are wall-clock means; standard deviations were below 4% for all measurements.

The **raw_fwd** row confirms the GHC Core result of Section 7.1: the typed and hand-rolled implementations are statistically indistinguishable.

Variant	Mean time	Diff. from baseline	Overhead identified
<code>fold-foldM</code>	3,632 μs	—	baseline
<code>fold-foldl'</code>	3,649 μs	+17 μs (noise)	fold structure: ≈ 0
<code>forward-only</code>	251 μs	−3,381 μs	<code>.backward()</code> + update
<code>flattenParams-x18</code>	< 1 μs	—	Generic traversal: < 0.03 μs
Typed forward $\times 18$		$18 \times 6.9 = 124 \mu s$	tensor arithmetic
<code>forward-only</code> − typed $\times 18$		$251 - 124 = 127 \mu s$	autograd graph build

Table 7.3: Overhead isolation for one dynamic epoch. The `forward-only` variant runs the full forward pass (including autograd graph construction) but never calls `.backward()`. Std devs: fold variants < 1%, `forward-only` 4%.

Typed lens vs. dynamic autograd

The *ParaLens* training loop (689 μs) is **5.3 $\times$ faster** than the dynamic autograd baseline (3,631 μs). Table 7.3 isolates each overhead source. The dominant cost (93%) is PyTorch’s C++ backward pass: the `forward-only` variant (forward pass only, no `.backward()` call) completes in 251 μs , leaving 3,381 μs attributable to reverse-mode AD. By contrast, the *ParaLens* backward is a chain of Haskell closures inlined at compile time, reducing to the same eight LibTorch FFI calls as the forward pass. Autograd graph construction (tape allocation and *Node* registration) accounts for a further 127 μs (3.5%); fold structure and parameter serialisation are negligible (< 0.03%).

Typed lens vs. HMatrix

HMatrix (390 μs) is 1.8 $\times$ faster on the 4×4 Iris matrices, where ATen’s multi-device dispatch [24] dominates over the underlying BLAS kernel [25]. For larger matrices or batch sizes the typed lens implementation is expected to match HMatrix and exceed it via LibTorch’s vectorised kernels.

7.6 Gradient Correctness Tests

The framework includes a test suite that cross-validates every primitive layer’s forward and backward pass against PyTorch autograd, providing numerical evidence that the hand-derived gradients are correct.

Method

Each test constructs identical inputs in both the typed *ParaLens* implementation and a dynamic reference model, runs one gradient-descent step via *runStep* with learning rate 1, and recovers the autograd gradient as the difference between old and new parameters. The typed gradient is compared against this reference with a maximum absolute tolerance of 10^{-4} . Because the typed backward pass is statically compiled Haskell rather than a traced

computation graph, any discrepancy would indicate an error in the hand-derived Jacobian, not a numerical accident.

Coverage

- *linear*: forward pass ($xW^\top$) and weight gradient ($\partial L/\partial W = \text{grad}^\top x$), batch size 1.
- *addLens*: forward pass (broadcast add) and bias gradient ($\partial L/\partial b = \sum_{\text{batch}} \text{grad}$), batch size 4.
- *sigmoid*: forward pass ($\sigma(x)$) and input gradient ($\text{grad} \cdot \sigma(x)(1 - \sigma(x))$), 10 values.
- *relu*: forward pass and Heaviside input gradient, 10 values.
- *convLens*: forward pass (unit stride, zero padding), and kernel gradient via the *im2col* batched matrix multiply, on a $1 \times 1 \times 7 \times 7$ input with a $2 \times 1 \times 3 \times 3$ kernel.
- *flatten*: forward reshape and backward reshape, verified to be exact inverses.
- *softMaxCELoss*: forward pass against the reference formula $-\frac{1}{B} \sum_{i,c} t_{ic} \log q_{ic}$; prediction gradient $\frac{1}{B}(q - t)$ verified against PyTorch autograd and against the explicit formula independently, confirming the per-batch normalisation factor.
- **Iris forward equivalence**: the full two-layer Iris model run on a batch of eight samples produces output identical to an equivalent dynamic hasktorch model given the same weights.
- **Iris epoch equivalence**: after one full training epoch (18 batches), the typed and dynamic models reach identical parameter values.

Relationship to Accuracy

Layer-level gradient tests are a stronger correctness signal than end-to-end accuracy: a model can converge to a non-trivial accuracy even with slightly incorrect gradients, whereas a Jacobian error above 10^{-4} would be caught directly. The epoch-equivalence test extends this to the composed training loop, confirming that composition via $(\bullet)$ and the *repara* update rule interact correctly across an entire epoch on real data.

Chapter 8

Conclusion

8.1 Summary

This thesis has instantiated the categorical framework of Cruttwell et al. [4] as a working Haskell library for gradient-based machine learning. The central abstraction, that every component of a training pipeline is a parametric lens carrying a forward pass and a gradient pass as a single composable unit, is realised here with full static verification. Tensor shapes, mini-batch sizes, compute devices, and element dtypes are all encoded as type-level parameters, so a misconfigured architecture is rejected by the type checker rather than failing at runtime or producing a silent numerical error.

The library extends the original Python proof-of-concept in eight concrete directions. First, all implementations are written in batched form from the ground up: every layer, loss function, and optimiser operates over a leading batch dimension b that is a type-level `Nat`, whereas the original Cruttwell et al. implementation processes a single example at a time. Second, all tensor dimensions are static: GHC’s `DataKinds` extension encodes every dimension as a type-level `Nat`, and type families compute output shapes for convolutions and pooling layers automatically, making spatial mismatches compile-time errors (Chapter 4). Third, device and dtype polymorphism are built into every model signature: a single *ParaLens*’ definition is valid on CPU or CUDA and in 32-bit or 64-bit floating point, with constraint synonyms such as *SaneDT* preventing invalid device/dtype combinations from type-checking. Fourth, the framework is extended beyond the MLP scope of the original paper to autoencoder architectures: encoder and decoder are each ordinary *ParaLens*’ pipelines that compose with $(\bullet)$ into a single end-to-end model, trained on MNIST images with MSE reconstruction loss. Fifth, scaled dot-product self-attention and its multi-head generalisation are implemented as *ParaLens*’ values with fully manually derived backward passes (including the rank-one Jacobian correction for the softmax non-linearity) extending the framework to the attention mechanism that underlies modern transformer architectures (Chapter 4). Sixth, residual skip connections are realised as a single higher-order combinator *skipPara*, built entirely from existing lens primitives

with no new axioms; the identity gradient path that ensures gradients reach early layers even when the learned branch saturates emerges automatically from the combinator structure, requiring no separate backward-pass derivation (Section 4.5). Seventh, type-level Peano naturals and typeclass induction are used to build networks of variable depth n whose parameter type is a fully concrete nested tuple at each n , preserving GHC’s ability to specialise and unbox the entire parameter structure and yielding zero overhead relative to a hand-written network of the same depth. Eighth, the zero-overhead property is verified directly by GHC Core inspection: compiling the lens-based forward pass and an equivalent hand-written forward pass with `-O2 -ddump-simpl` produces structurally identical worker functions, confirming that the entire *ParaLens* abstraction: composition, reparametrisation, the van Laarhoven $\forall$; is absent from the optimised output (Section 7.1).

8.2 Future Work

Several directions remain open for future investigation.

Recurrent architectures. Recurrent networks maintain hidden state across time steps, which does not fit the stateless composition pattern of $(\bullet)$ directly. One avenue is to treat the hidden state as an additional parameter, making the recurrence an instance of *repara* on the state wire. Whether this recovers standard BPTT or requires a categorical extension remains an open question.

Investigation of reverse pass deduplication It is conceivable that with sufficient massaging of inlining pass order, GHC can spot duplicated calls to forward passes (see Section 8.3) in lens setters. A Core-level experiment makes this concrete: specialising a composed attention setter and inspecting the optimised worker shows the scaled $QK^\top$ score and its softmax evaluated twice, once for the output the downstream layer consumes and once inside attention’s reverse pass. Eliminating this measured recomputation is a promising avenue that should fix a key limitation of the existing framework.

Discrete and probabilistic CRDCs. The Cruttwell et al. framework is defined for any CRDC, not just smooth Euclidean spaces. Instantiating the Haskell library for a discrete CRDC (e.g., a category of Boolean circuits) or a probabilistic one would generalise gradient-based learning beyond the real-valued setting and test the abstraction boundaries of the current design.

8.3 Limitations

Several limitations of the current design are worth stating directly.

Forward-pass recomputation in the backward pass. The Van Laarhoven lens representation encodes the forward pass in the getter and the backward pass in the setter as a single higher-rank function. This means the setter receives the original input and the incoming gradient, but not the intermediate activations the getter computed along the way. For simple layers such as *linear* this is costless, but *selfAttention* must recompute all six intermediate tensors (Q , K , V , the score matrix, softmax weights, and attended output) by calling *runFwd* a second time, performing two forward passes per training step. The root cause is structural: the lens representation provides no channel for the getter to pass cached state to the setter. Replicating the autograd remedy (an explicit computation graph whose nodes retain their outputs) would require encoding the activation cache in the parameter type, significantly complicating the type signatures.

Global optimiser state and step-dependent updates. The *repara* mechanism carries per-layer optimiser state on the vertical parameter wire, which handles momentum buffers and accumulated gradient statistics cleanly. It does not, however, accommodate state that is global across layers or that evolves with a step counter independent of any individual layer. Adam’s bias-correction terms $\hat{m} = m/(1 - \beta_1^t)$ and $\hat{v} = v/(1 - \beta_2^t)$ require the current step t , which the current implementation omits (Section 6.3): including t would require adding a step counter to every layer’s optimiser state or breaking the uniform *repara* interface. The same difficulty applies to learning-rate schedules, gradient clipping by global norm, and weight decay as a separate regularisation term.

Stateful and stochastic layers. Several standard layer types do not fit the deterministic, stateless *ParaLens'* model. Dropout requires sampling a random mask during the forward pass and retaining it for the backward pass, which demands an *IO* or *State* monad absent from the current type. Batch normalisation maintains running mean and variance statistics that are updated during training but frozen at inference time, requiring a training-mode flag that is not encoded in *ParaLens'*. Layer normalisation is deterministic and fits cleanly; the others do not. Any model that relies on dropout or batch normalisation therefore cannot be expressed in the current framework without a more substantial extension to the core type.

Type-checker scaling. The type-level depth parameter uses Peano naturals and typeclass induction. At $n = 3$, *StackedN 3 m* reduces to $(m, (m, m))$ at compile time, which GHC can fully specialise and unbox. GHC’s constraint solver is known to exhibit quadratic or worse behaviour on deeply nested tuple types and long typeclass resolution chains in general. In practice, compilation times at $n = 50$ remained acceptable, and Core size grows linearly with depth (Section 7.1), suggesting that the typeclass resolution cost is

also approximately linear for this particular induction structure. For substantially deeper networks—a 50-layer ResNet with multiple sub-layers per block, for instance—compile-time scaling is plausible but untested beyond the depths evaluated here.

Inliner budget and the zero-overhead guarantee. The zero-overhead property depends on GHC’s simplifier fully inlining and specialising the $\forall f \circ Functor f \Rightarrow$ quantifier at each call site and eliminating all the tuple isomorphisms (*swapFst*, *rotate*, *rightLens*) that each $(\bullet)$ introduces. GHC’s simplifier operates within a finite tick budget `-fsimpl-tick-factor`; each $(\bullet)$ composition adds several reduction steps, so a sufficiently deep network could in principle exhaust this budget. In practice, the property has been verified at $n = 1$, $n = 20$ (1,800 lines of Core), and $n = 50$ (4,900 lines of Core), with Core size growing linearly and the lens abstraction absent from the output at every depth tested (Section 7.1). This suggests the guarantee holds for any practically realisable network; should a sufficiently deep model ever approach the default limit, raising `-fsimpl-tick-factor` explicitly is a straightforward mitigation.

Type error ergonomics. Even when the type system catches a mistake, the resulting error message may be difficult to interpret. Using local aliases to simplify the types, passing a *Tensor* `[1, 10]` where *Tensor* `[1, 4]` is expected:

```
type Tensor s = T.Tensor DV DT s
type MMP o i = L.MMP DV DT o i
foo :: ParaLens' (MMP 6 4) (Tensor [b, 4]) (Tensor [b, 6])
foo = matMulLens
x :: Tensor [1, 6]
x = view foo (p :: MMP 6 4, oops :: Tensor [1, 10])
```

nonetheless produces:

```
* Couldn't match type '10' with '4'
   arising from a functional dependency between:
     constraint 'mtl-2.3.1:Control.Monad.Reader.Class.MonadReader
               (MMP 6 4, Tensor [1, 4]) ((->) (MMP 6 4, Tensor [1, 10]))'
   arising from a use of 'view'
   instance 'mtl-2.3.1:Control.Monad.Reader.Class.MonadReader
            r ((->) r)'
   at <no location info>
* In the expression:
   view foo (p :: MMP 6 4, oops :: Tensor [1, 10])
```

In an equation for 'x':

```
x = view foo (p :: MMP 6 4, oops :: Tensor [1, 10])
```

The core diagnosis (*Couldn't match type '10' with '4'*) is correct, but it is buried inside a functional-dependency trace through *mtl's MonadReader*, an artefact of the Van Laarhoven encoding that a user should not need to understand. Errors from failed type family unification in *ConvSideCheck* or *StackedN* are more verbose still. Using the framework's own runner (*runFullModel*) yields cleaner messages, but the ergonomics cost remains and affects accessibility for practitioners unfamiliar with GHC's type system.

Benchmark scope. All performance results are from a CPU-only machine with toy models. The $5.3\times$ speedup over dynamic autograd is measured on a two-layer Iris MLP with 1,527 parameters and batch size 8. On GPU workloads with large models the cost of PyTorch's autograd tape construction shrinks relative to the tensor arithmetic itself, so the advantage is likely smaller or absent at scale. The claim that the typed lens implementation will match HMatrix for larger matrices (Section 7.5) is plausible given the per-call FFI overhead analysis, but is asserted rather than measured.

8.4 Broader Impact

One key benefit of this work is the promotion of ML configuration errors (e.g. shape mismatches, invalid device/dtype combinations, architectural depth mistakes) from silent runtime failures to compile-time rejections. This manifested concretely during development: porting an untyped prototype to the typed implementation exposed asymmetric index errors in several backward passes that had gone undetected because tests used equal input and output dimensions; the type system rejected them immediately.

The guarantees are nonetheless narrow: they address gradient mechanics and tensor book-keeping, not deployment behaviour. A well-typed network trained on biased data remains biased. There is also an access barrier: the safety properties described here require Haskell fluency, gating the benefits behind a significant learning curve and reinforcing existing stratification in who can audit ML infrastructure. A productive long-term direction is to export these ideas to mainstream frameworks via type-level extensions to Python or automatic generation of typed interfaces, making compile-time shape safety accessible without a change of language.

Chapter 9

Declarations

9.1 Generative AI

I acknowledge the use of Anthropic’s Claude Sonnet 4.6 (<https://www.claude.ai/>) in the writing of this thesis. Claude was used for: assistance in drafting the initial testing suite, single-run scripts for visualising training, assistance in debugging the core implementation of the library and suggestions for improving the phrasing and legibility of the report.

I confirm that no content generated by AI has been presented as my own work without significant verification and revision, and that I have taken full responsibility for the content of this thesis.

9.2 Ethical Considerations

This project involves no human subjects, animal studies, personal data, or sensitive information. All datasets used (Iris, MNIST) are established public benchmarks with no associated privacy concerns. The library is released as open-source software under a permissive licence. No conflicts of interest arise from this work. No formal ethical approval was required or sought.

9.3 Sustainability

All training experiments were conducted on a single CPU-only laptop; no cloud compute, GPU clusters, or high-performance computing facilities were used. The models are intentionally small (1,527 parameters for the MNIST CNN) and training runs completed in minutes to hours rather than days. A central result of the thesis is that the typed-lens framework is $5.3\times$ more efficient than PyTorch’s dynamic autograd on the benchmark tasks (Section 7.5), which translates directly to reduced energy consumption for equivalent workloads. No new hardware was purchased for this project.

Appendix A

Hasktorch Typed-API Bug Workarounds

Two bugs in the hasktorch typed API (version 0.2.0.0, LibTorch 2.x) affect the convolutional backward pass implemented in `Static/Layers.hs`. Both were discovered during development of the *convLens* backward pass and are worked around in `Static/Bugfix.hs`.

A.1 Transposed Convolution: Swapped ConvSideCheck Arguments

The hasktorch typed binding for *convTranspose2d* applies the *ConvSideCheck* constraint with the input and output spatial dimensions swapped relative to the ATen semantics. The library checks

$$\text{ConvSideCheck } \textit{outputSize} \textit{ kernelSize } \textit{stride padding inputSize}$$

where the correct direction is from input to output:

$$\text{ConvSideCheck } \textit{inputSize kernelSize stride padding outputSize}.$$

For 1×1 kernels with unit stride and zero padding, the two checks happen to be equivalent (the constraint reduces to $\textit{outputSize} = \textit{inputSize}$ in both cases), so the bug is invisible at that configuration. The hasktorch test suite exercises only 1×1 kernels for *convTranspose2d*, which is why the error was not caught. For all other kernel sizes the swapped constraint rejects valid calls and accepts invalid ones.

The corrected binding in *Bugfix.hs* swaps the size arguments to *ConvSideCheck* to match the ATen semantics, and was verified by cross-checking output shapes against dynamic (untyped) calls on 3×3 and 4×4 kernels.

A.2 im2col: Unimplemented Castable Instance

The hasktorch typed API does expose *im2col*, but its binding passes Haskell tuples where the underlying ATen FFI expects lists. The call typechecks because a

Castable (Int, Int) (ForeignPtr IntArray) instance exists ¹, but that instance delegates to *makeTuple2*, which is defined as: ²

```
makeTuple2 = error "makeTuple2 is not implemented."
```

Any call to the hasktorch *im2col* binding therefore compiles successfully but throws a Haskell runtime error unconditionally.

The corrected binding bypasses the original *Castable (Int, Int) (ForeignPtr IntArray)* by converting each pair to a two-element list before the cast, and thus using

Castable [Int] (ForeignPtr IntArray):

```
im2col
:: U.Tensor → (Int, Int) → (Int, Int) → (Int, Int) → (Int, Int)
→ U.Tensor
im2col self (kh, kw) (dh, dw) (ph, pw) (sh, sw) =
  unsafePerformIO $ (cast5 im2col_tllll) self [kh, kw] [dh, dw] [ph, pw] [sh, sw]
```

The fix is otherwise identical to the original binding. The correctness of the output shape $[batch, inC * kH * kW, oH * oW]$ was verified against reference outputs on several kernel, padding, and stride configurations.

¹<https://github.com/hasktorch/hasktorch/blob/043552377932093d36e10d46e8b45aaa44225f2d/libtorch-ffi/src/Torch/Internal/Cast.hs#L237>

²<https://github.com/hasktorch/hasktorch/blob/043552377932093d36e10d46e8b45aaa44225f2d/libtorch-ffi/src/Torch/Internal/Class.hs#L23>

Bibliography

1. LeCun Y, Bengio Y, and Hinton G. Deep learning. *Nature* 2015; 521:436–44
2. Shiebler D, Gavranović B, and Wilson P. Category Theory in Machine Learning. 2021. arXiv: 2106.07032 [cs.LG]. Available from: <https://arxiv.org/abs/2106.07032>
3. Plaut DC, Nowlan SJ, and Hinton GE. Experiments on Learning by Back Propagation. 1986. Available from: <https://api.semanticscholar.org/CorpusID:15150815>
4. Cruttwell GSH, Gavranović B, Ghani N, Wilson P, and Zanasi F. Categorical Foundations of Gradient-Based Learning. 2021. arXiv: 2103.01931 [cs.LG]. Available from: <https://arxiv.org/abs/2103.01931>
5. Fong B, Spivak DI, and Tuyéras R. Backprop as Functor: A compositional perspective on supervised learning. 2019. arXiv: 1711.10455 [math.CT]. Available from: <https://arxiv.org/abs/1711.10455>
6. Fong B and Johnson M. Lenses and Learners. 2019. arXiv: 1903.03671 [cs.LG]. Available from: <https://arxiv.org/abs/1903.03671>
7. Elliott C. The simple essence of automatic differentiation. 2018. arXiv: 1804.00746 [cs.PL]. Available from: <https://arxiv.org/abs/1804.00746>
8. Pickering M, Gibbons J, and Wu N. Profunctor Optics: Modular Data Accessors. *The Art, Science, and Engineering of Programming* 2017 Apr; 1. DOI: 10.22152/programming-journal.org/2017/1/7. Available from: <http://dx.doi.org/10.22152/programming-journal.org/2017/1/7>
9. Kmett E. Control.Lens.Lens — lens and related combinators. Accessed: 2026-01-20. 2025. Available from: <https://hackage-content.haskell.org/package/lens-5.3.5/docs/src/Control.Lens.Lens.html#lens>
10. Foster JN, Greenwald MB, Moore JT, Pierce BC, and Schmitt A. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View Update Problem. *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM Press, 2005 :233–46. DOI: 10.1145/1040305.1040325

11. Bancilhon F and Spyrtos N. Update semantics of relational views. *ACM Transactions on Database Systems* 1981; 6:557–75
12. Peyton Jones S. Lenses: Compositional Data Access and Manipulation. Talk at Haskell eXchange 2013. Video recording archived by the Internet Archive; accessed 2026-01-20. 2013. Available from: <https://archive.org/details/lenses-compositional-data-access-and-manipulation-simon-peyton-jones-at-haskell->
13. Peyton Jones S et al. Haskell 2010 Language Report. Haskell.org. 2010. Available from: <https://www.haskell.org/onlinereport/haskell2010/>
14. The Glasgow Haskell Compiler Team. Functor — `Control.Monad`. Accessed: 2026-01-20. 2025. Available from: <https://hackage-content.haskell.org/package/base-4.22.0.0/docs/Control-Monad.html#t:Functor>
15. The GHC Developers. `GHC.Internal.Data.Functor.Identity` — Documentation. <https://hackage-content.haskell.org/package/ghc-internal-9.1401.0/docs/src/GHC.Internal.Data.Functor.Identity.html#Identity>. Accessed: 2026-01-20, version ghc-internal-9.1401.0. 2026
16. The GHC Developers. `GHC.Internal.Data.Functor.Const` — Documentation. <https://hackage-content.haskell.org/package/ghc-internal-9.1401.0/docs/src/GHC.Internal.Data.Functor.Const.html#Const>. Accessed: 2026-01-20, version ghc-internal-9.1401.0. 2026
17. Breitner J, Eisenberg RA, Peyton Jones S, and Weirich S. Safe zero-cost coercions for Haskell. *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*. ICFP '14. Association for Computing Machinery, 2014 :189–202. DOI: 10.1145/2628136.2628141
18. Campbell H. Grenade: Dependently Typed Neural Networks in Haskell. <https://hackage.haskell.org/package/grenade>. Haskell package, Hackage; last substantively updated 2022. 2016
19. Ruiz A et al. `hmatrix`: Numeric Linear Algebra and Matrix Computations. <https://hackage.haskell.org/package/hmatrix>. Haskell package, Hackage
20. Le J. `backprop`: Heterogeneous automatic differentiation. <https://hackage.haskell.org/package/backprop>. Haskell package, Hackage. 2018
21. Kingma DP and Ba J. Adam: A Method for Stochastic Optimization. 2017. arXiv: 1412.6980 [cs.LG]. Available from: <https://arxiv.org/abs/1412.6980>
22. He K, Zhang X, Ren S, and Sun J. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. 2015 Feb. arXiv: 1502.01852 [cs.CV]

23. O’Sullivan B. criterion: Robust, reliable performance measurement and analysis. <https://hackage.haskell.org/package/criterion>. Haskell package, Hackage. 2009
24. Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, Desmaison A, Köpf A, Yang E, DeVito Z, Raison M, Tejani A, Chilamkurthy S, Steiner B, Fang L, Bai J, and Chintala S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. eprint: **arXiv:1912.01703**
25. Frison G, Sartor T, Zanelli A, and Diehl M. The BLAS API of BLASFEO: optimizing performance for small matrices. 2019. DOI: **10.1145/3378671**. eprint: **arXiv:1902.08115**